

Should You Even Train a Neural Network?

A Forecasting-Based Early Rejection Strategy under Budget Constraints

Santiago Ramírez Castañeda

Universidad de La Sabana

Faculty of Engineering

M.Sc. in Applied Analytics

Thesis Advisor: **Felix Mohr**

November 11, 2025



Abstract

Neural networks frequently dominate unstructured learning tasks but often fail to outperform simpler, non-neural baselines on tabular data—where ensembles such as Gradient Boosted Trees remain superior under equal budgets. This work introduces ShouldYouEvenNN, a forecasting-based decision framework that determines whether continued neural training is rational given early evidence and computational constraints. Instead of performing exhaustive Neural Architecture Search (NAS), the method reframes model exploration as a continuation decision problem. The framework integrates three core mechanisms: (i) a shape-aware morphing forecaster that extrapolates early batch-level validation signals through a smooth blend of rational, sigmoid, and linear functions; (ii) a probabilistic scoring layer estimating the likelihood that a neural candidate will surpass a strong non-neural baseline; and (iii) a cost-aware expected-utility criterion that decides whether training should proceed. Experiments across seventy-one OpenML datasets show that ShouldYouEvenNN correctly identified neural-worthy tasks in over half of the cases, achieving validation accuracies within 5–6 percentage points of exhaustive NAS results (LCBench) while reducing runtime significantly.

These findings demonstrate that early learning dynamics can provide sufficient statistical evidence to govern NAS decisions, transforming neural exploration from blind optimization into rational inference. ShouldYouEvenNN thus offers a principled step toward evidence-based automation, enabling resource-efficient model discovery that decides not only how to train, but whether training is worth it.

Chapter 1

Introduction

This chapter outlines the motivation for the study and frames the central question: *when is training a neural network worth the computational cost?* It summarizes the limitations of conventional Neural Architectural Search [NAS] methods, the dominance of non-neural baselines, and the rationale for reframing architecture search as a decision problem.

Neural networks have transformed modern machine learning, achieving state-of-the-art performance across diverse domains such as vision, language, and forecasting. Yet their design and training remain expensive and limiting. The vastness of possible architectures, hyperparameters, and learning dynamics makes neural exploration a computational gamble—especially when each evaluation demands full convergence to reveal its true potential.

Paradoxically, in structured or tabular domains -the setting of most practical data analytics- non-neural methods still dominate (Shwartz-Ziv & Armon, 2022). Ensemble-based models such as Gradient Boosted Trees and Random Forests routinely outperform neural networks under comparable optimization budgets, offering strong generalization with minimal tuning and far lower computational cost (Borisov et al., 2022; Grinsztajn et al., 2022; Mohr & van Rijn, 2023). This persistent asymmetry raises a fundamental and largely unaddressed question: *when is training a neural network actually worth it?*

Automated Machine Learning (AutoML) frameworks have made great strides toward automation and efficiency. They identify strong baseline models through algorithm selection and hyperparameter optimization, often without requiring deep expertise (Elsken et al., 2019; Mohr & Wever, 2023). However, while AutoML systems streamline the discovery of high-performing

non-neural configurations, Neural Architecture Search (NAS) extends this automation to deep models—frequently at the expense of massive computational budgets. Based on our initial review, most NAS research focuses on accelerating search within the neural family, rather than determining whether neural architectures are even competitive for the problem at hand.

Recent work has shown that early training dynamics—partial learning curves, gradient signals, or batch-level metrics—can reveal predictive information about a model’s eventual performance (Domhan et al., 2015; Egele et al., 2024; Ren et al., 2022). These findings suggest that evaluating a candidate’s promise does not necessarily require full training. What remains missing is a principled mechanism to *act* on this information: a decision rule that weighs early neural evidence against efficient non-neural baselines.

This thesis introduces *ShouldYouEvenNN*, a forecasting-driven decision framework designed to answer precisely that question. Instead of treating NAS as a blind search for optimal architectures, *ShouldYouEvenNN* formulates it as a rational decision process under computational uncertainty. Each neural candidate is allocated a uniform and limited amount of training effort, from which early validation signals are extrapolated to forecast long-term accuracy. These forecasts are then evaluated probabilistically against a reference baseline through an expected-utility criterion that quantifies whether continued neural training is justified.

By transforming early learning behavior into actionable evidence, *ShouldYouEvenNN* bridges the gap between AutoML efficiency and NAS exploration. The remainder of this work situates *ShouldYouEvenNN* within the broader landscape of automated model design and selection. The next section reviews the evolution of AutoML systems, emphasizing how they streamline preprocessing, algorithm selection, and hyperparameter optimization to identify competitive non-neural baselines efficiently. It then contrasts these approaches with NAS methods, which aim to automate neural design but remain constrained by high computational costs. Finally, it surveys recent advances in early-termination and learning-curve forecasting strategies that seek to reduce such costs—developments that directly motivate the decision-oriented formulation adopted by *ShouldYouEvenNN*.

Chapter 2

Background and Related Work

This chapter situates the research within the existing literature on AutoML, NAS, and early-signal forecasting. It traces the progression from automated model selection to architecture search, emphasizing efficiency and decision-making under uncertainty.

2.1 Automated Machine Learning and Model Selection

AutoML aims to make advanced modeling broadly accessible by automating the stages of data preprocessing, model selection, and hyperparameter optimization. By formalizing these steps as a joint optimization problem, AutoML systems remove much of the trial-and-error traditionally required in machine learning workflows (Baratchi et al., 2024). A central theoretical formulation within AutoML is the *Combined Algorithm Selection and Hyperparameter Optimization* (CASH) problem (Thornton et al., 2013), defined as

$$A^*, \lambda^* \in \arg \min_{A^{(j)} \in \mathcal{A}, \lambda \in \Lambda^{(j)}} \frac{1}{k} \sum_{i=1}^k \mathcal{L} \left(A_{\lambda}^{(j)}, D_{\text{train}}^{(i)}, D_{\text{valid}}^{(i)} \right), \quad (2.1)$$

where $\mathcal{A}$ denotes the set of learning algorithms, $\Lambda^{(j)}$ their hyperparameter spaces, and $\mathcal{L}$ a validation loss function. This definition formalizes the automated search for the optimal algorithm–configuration pair (A^*, λ^*) that minimizes validation error over k folds.

Early AutoML frameworks approached the CASH problem using Bayesian optimization and ensemble strategies to traverse algorithm and hyperparameter spaces efficiently (Nguyen et al., 2022; van Hoof & Vanschoren, 2021; Z. Wang et al., 2017; Wu et al., 2020). Subsequent gen-

erations integrated meta-learning and evolutionary search (Jurado & Vanschoren, 2022; Salehin et al., 2024), balancing exploration and exploitation to reduce runtime while maintaining accuracy. Recent work, by Mohr and Wever (2023) demonstrated that much of this complexity can be reduced through separable optimization: optimizing preprocessing and modeling stages independently yields comparable performance to full CASH solvers, revealing that simplicity and transparency often outperform algorithmic intricacy.

AutoML’s evolution reflects a clear principle: efficient automation can replace exhaustive experimentation without sacrificing generalization. This principle provides the foundation upon which neural automation—specifically, NAS—was later built.

2.2 Neural and Non-Neural Baselines on Tabular Data

Despite the success of neural networks in unstructured domains such as vision, text, and speech, their dominance does not extend to tabular or structured data. Comprehensive reviews and benchmarks consistently show that tree-based ensembles, particularly Gradient Boosted Decision Trees (GBDTs) and Random Forests, outperform feedforward neural networks on the majority of tabular datasets (Borisov et al., 2022; Grinsztajn et al., 2022; Mohr & van Rijn, 2023). The reason is both architectural and statistical: GBDTs naturally handle heterogeneous feature scales, categorical variables, and missing values without extensive preprocessing, whereas neural networks require larger datasets, hyperparameter sensitivity, and extensive regularization to achieve comparable results.

Nonetheless, this superiority is not absolute. McElfresh et al. (2023) showed that deep networks can outperform GBDTs in high-dimensional and interaction-heavy datasets, provided sufficient data and regularization. Gardner et al. (2022) found that ensembles often yield better subgroup robustness, while neural networks can capture nonlinear feature hierarchies inaccessible to additive models. Also, Gorishniy et al. (2021) found no universally superior solution when testing GBDTs versus models such as ResNet or Transformers. The tension between flexibility and efficiency persists: neural models can, in principle, surpass non-neural ones, but only under favorable conditions that justify their cost. This tension motivates the central question underlying this work.

2.3 Neural Architecture Search and the Problem of Efficiency

NAS extends the principles of AutoML to the design of neural networks. Its goal is to discover architectures that achieve the lowest validation loss after training. However, this objective exposes NAS’s central inefficiency: evaluating the true quality of a neural architecture requires fully training it to convergence, which makes the process computationally expensive.

NAS approaches have relied on reinforcement learning and evolutionary algorithms (Mendoza et al., 2016; Zoph & Le, 2016), but computational demands were immense—training tens of thousands of networks from scratch on massive datasets. Later works introduced weight sharing and one-shot models (Jin et al., 2019; Lu et al., 2024), as well as differentiable search spaces (Liu et al., 2024; W. Wang & Li, 2025), drastically reducing redundant computation. Hybrid methods such as Lin et al. (2024), Sakamoto et al. (2023), and Zheng et al. (2021) further improved exploration through multi-armed bandits and task transfer.

Still, scalability and reproducibility remain open issues (Mohr & van Rijn, 2022). Even efficient NAS systems are often constrained by the cost of evaluating candidates to convergence, making them impractical for small or medium-sized problems. Ren et al. (2022) and Garcia-Garcia et al. (2025) proposed progressively expanding training budgets to reduce wasted computation, while Jurado and Vanschoren (2022) introduced multi-fidelity strategies that evaluate partial models before committing additional resources. Despite these advances, most approaches still assume that some form of full training will ultimately be completed. The missing element is a mechanism to determine whether such training should even begin.

2.4 Early-Training Signals and Forecasting in NAS

Research over the past decade has shown that the early stages of neural network training—sometimes just a few epochs or even mini-batches—contain information predictive of the model’s eventual quality. Learning-curve extrapolation approaches such as those proposed by Domhan et al. (2015) and Swersky et al. (2014) fit parametric forms (exponential, logistic, or rational) to partial validation trajectories to estimate asymptotic performance. Subsequent analyses by White et al. (2021), Yan et al. (2021), and Wistuba and Pedapati (2020) extended these methods to large NAS bench-

marks, confirming that forecast accuracy from early signals correlates strongly with final validation outcomes.

Beyond curve fitting, several works leverage static or gradient-based indicators. Wen et al. (2019) introduced zero-cost proxies derived from Jacobian conditioning, while Ru et al. (n.d.) exploited gradient norms to anticipate overfitting tendencies. Probabilistic and Bayesian formulations (Adriaensen et al., 2023; Wei et al., 2020) incorporate uncertainty into these predictions, pruning unpromising architectures early in the search. Similar ideas have appeared in AutoML, where partial evaluations and meta-surrogates are used to approximate cross-pipeline performance under time constraints (Aach et al., 2023; Guan et al., 2025; Solís-Martín et al., 2025).

The same rationale has influenced hyperparameter optimization. Egele et al. (2023, 2024) demonstrated that one-epoch results can approximate long-run outcomes with minimal accuracy loss, reducing training time substantially. These findings reinforce that early metrics are not merely transient diagnostics but meaningful predictors of future performance.

Forecasting thus became a central tool for efficiency-driven NAS. Bayesian curve-fitting, Gaussian-process surrogates, and polynomial extrapolations (Adriaensen et al., 2023; Mebirouk et al., 2024; Patel et al., 2024) enable the rapid ranking of architectures using partial learning data. More recent studies (Nandal et al., 2025; W. Wang & Li, 2025) incorporate uncertainty quantification to adaptively balance exploration and exploitation.

However, most existing NAS approaches employ forecasting solely as an auxiliary ranking signal—rarely as an explicit decision mechanism. The question of whether neural exploration should continue at all remains largely heuristic. This gap motivates decision-oriented frameworks such as *ShouldYouEvenNN*, which interpret forecasts not just as estimates of performance but as actionable evidence for rational continuation under limited computational budgets.

Summary. Together, these sections reveal a persistent gap: existing NAS methods optimize *what* to train but not *whether* to train, motivating the decision-oriented formulation of *ShouldYouEvenNN*.

Chapter 3

Problem Definition and Motivation

This chapter defines the conceptual gap between neural and non-neural automation. It formalizes the decision question—under what conditions neural training becomes rational—and introduces the expected-utility formulation that will guide later chapters.

The literature reviewed in the previous chapter reveals a clear imbalance in how automation has evolved across learning paradigms. While AutoML has successfully optimized model selection and hyperparameters across non-neural families, NAS remains heavily focused on structural optimization rather than strategic decision-making. That is, NAS seeks the best neural architecture *assuming* neural models are worth training, whereas AutoML frameworks often demonstrate that simpler, non-neural models suffice for most real-world problems (Borisov et al., 2022; Grinsztajn et al., 2022). This assumption of neural worthiness produces a systematic inefficiency: NAS frameworks spend vast computational budgets exploring architectures that, for many datasets, cannot outperform ensemble-based baselines.

The challenge, therefore, is not merely to accelerate NAS, but to determine when neural training is rational in the first place. Multi-fidelity and early-termination techniques (Egele et al., 2023, 2024; Garcia-Garcia et al., 2025; Jurado & Vanschoren, 2022; Ren et al., 2022) mitigate cost by pruning unpromising runs; yet these strategies remain reactive—they halt low-performing candidates but do not decide whether neural exploration itself is justified. The field lacks a unified mechanism for comparing early neural progress against established non-neural standards to decide whether additional neural optimization is worthwhile.

As seen in Section 2.4, evidence suggests that once a neural architecture has undergone a

limited amount of training, its partial learning curve already reflects its early validation trajectory. From this partial evidence, one can *forecast* its limit performance, denoted V_{NN}^{limit} . Let V_{base} denote the validation performance of a strong non-neural baseline, such as a Gradient-Boosted ensemble.

The central question can then be posed as a binary decision:

Does there exist any parameterization θ such that $V_{NN}^{\text{limit}}(\theta) > V_{\text{base}}$?

In practice, since V_{NN}^{limit} is itself an estimate, one must rely on its forecasted approximation derived from early learning behavior to judge whether continued neural optimization is rational. Thus, the problem shifts from asking *What is the optimal architecture?* to the more pragmatic question: *Is there a neural candidate worth training at all?*

This gap motivates a reframing of NAS as a *decision problem under uncertainty*. Rather than searching exhaustively within a neural space, is possible to use the partial training evidence to estimate if neural models can surpass non-neural baselines. Even more, also to weigh that probability against the expected computational cost of continuing. This is based on the net value of information (NVI) concept, which aims to weigh the anticipated benefit of obtaining additional evidence against the related cost or delay (Heckerman et al., 2013), this could be explained as:

$$\text{NVI}(S) = \mathbb{E}[U(S)] - C(S) \tag{3.1}$$

being S a sequence of steps to be performed, U the utility after executing S and C the associated cost with S . This view treats model training not as a guaranteed investment but as a stochastic process whose continuation should depend on its expected return and associated cost.

In this sense, *ShouldYouEvenNN* does not compete with NAS—it governs it. It serves as a decision mechanism capable of halting neural exploration when the evidence suggests that the expected payoff no longer justifies the computational cost.

This conceptual shift transforms neural architecture search from an open-ended optimization procedure into a disciplined, empirically-driven decision process, aligning model discovery with the principles of rational resource allocation.

3.1 Research Question

The core question behind this research is: *For supervised classification tasks, can early-training signals and lightweight evaluation strategies be effectively used to predict whether neural network architectures will achieve competitive performance compared to strong non-neural models, enabling a fast and resource-efficient NAS framework?*

3.2 Research Objectives

3.2.1 General Objective

To develop a fast and resource-efficient framework capable of determining whether neural networks are likely to yield competitive performance compared to strong non-neural models, by leveraging early training information and lightweight evaluation strategies.

3.2.2 Specific Objectives

- Develop and evaluate a *shape-aware morphing forecaster* capable of approximating the limit validation performance of neural networks from early batch-level signals.
- Develop a lightweight, cost-aware decision mechanism that, given a predefined pool of neural architectures and training hyperparameters, uses early forecasting evidence to decide which configurations should continue training (i.e., extend their learning curves) and which should be discarded, until a single candidate remains whose continuation is empirically justified relative to strong non-neural baselines.
- Empirically validate the overall *ShouldYouEvenNN* framework by benchmarking its selected neural architectures against random search results such as LCBench, assessing whether comparable performance can be achieved with substantially lower computational cost.

Summary. This framing transforms NAS from a brute-force optimization exercise into a resource-allocation problem, setting the theoretical foundation for the proposed methodology.

Chapter 4

Methodology

4.1 Overview of the *ShouldYouEvenNN* Framework

ShouldYouEvenNN is a decision-oriented framework that reframes NAS as a rational continuation problem under limited budgets. Instead of exhaustively optimizing architectures, the framework evaluates whether further neural training is *worth pursuing* based on early training evidence. Algorithm 1 provides a schematic view of the overall process.

Algorithm 1 High-level Flow of *ShouldYouEvenNN*

Require: Dataset D , baseline metric g

Ensure: Decision on whether neural training is worthwhile

```
1:  $P \leftarrow \text{InitializePopulation}(D)$ 
2: repeat
3:    $\text{TrainCandidates}(P)$  ▷ one-epoch, batch-logged
4:    $\text{ForecastPerformance}(P)$  ▷ morphing forecaster
5:    $\text{ScoreCandidates}(P)$  ▷  $p(\text{beat } g)$  from forecast
6:    $\text{PruneAndDrop}(P)$  ▷ discard hopeless/stagnant; keep elites/incubators
7:    $P \leftarrow \text{SpawnNewCandidates}(P)$  ▷ random + breed&mutate from survivors
8:    $U \leftarrow \text{EvaluateExpectedUtility}(P, g)$ 
9: until  $U > 0$  or time budget exhausted
10:  $\text{RefineAndValidate}(P, g)$  ▷ confirm candidate surpasses  $g$ 
```

The system operates in sequential phases that will be detailed in the section 4.2, but this is a general overview:

1. **Candidate initialization.** (I. 1) A bounded and configurable search space of multilayer perceptrons (MLPs) defines the structural and hyperparameter configurations from which a number of neural candidates are instantiated randomly.

2. **Batch-level training.** (l. 3) Each candidate undergoes short, small training rounds on progressively larger subsets of data, logging batch-level validation metrics that describe its early learning trajectory.
3. **Forecasting.** (l. 4) Early validation curves are extrapolated using shape-aware morphing of rational, sigmoid, and linear models, yielding a forecasted accuracy and its confidence interval.
4. **Probabilistic scoring.** (l. 5) Forecasts are transformed into a probability of surpassing a strong non-neural baseline using a hybrid estimator, producing a normalized score per candidate.
5. **Evolutionary refinement.** (l. 7) High-probability candidates are recombined and mutated to form the next generation, while unpromising ones are discarded, ensuring both diversity and efficiency.
6. **Expected-utility decision.** (l. 8) At the end of each generation, the top candidate is evaluated through an expected-utility criterion that weighs potential forecasted gain against remaining computational cost.
7. **Refinement and final decision.** (l. 10) If the utility threshold is met, the corresponding architecture is refined under early-stopping; otherwise, a fallback refinement confirms whether neural networks are uncompetitive for the dataset.

The algorithm can be seen in more detail at Appendix 8.1.

Generational structure. ShouldYouEvenNN operates as an iterative evolutionary loop composed of discrete generations. Each generation performs one complete round of training, forecasting, scoring, and pruning over the current population of neural candidates, after which new architectures are introduced through mutation or random resampling. Within a generation, every neural architecture evaluated during the search is represented as a *candidate*, and each undergoes a single training cycle—one epoch of mini-batch updates on a controlled subset of the data—before its validation trajectory is used for forecasting. This synchronized one-epoch rhythm ensures that all candidates are evaluated under comparable computational conditions and that progress across

generations can be measured as a sequence of consistent, lightweight updates rather than as full training runs.

4.2 Core Components

4.2.1 Candidate Initialization

The initial population $\mathcal{P}_0$ is constructed by sampling a fixed number N of neural architectures from the bounded search space Θ . A global random seed initializes synchronized NumPy, PyTorch, and Python generators to ensure reproducibility.

Each candidate is obtained by drawing an architecture

$$\theta_i \sim \mathcal{S}(\Theta, \xi),$$

where $\mathcal{S}$ is the stochastic sampler and ξ the shared RNG state. The sampler selects depth, width pattern, activation, learning rate, dropout, optimizer, batch size, and regularization from the ranges in Table 5.1. Formally,

$$\mathcal{P}_0 = \{C_1, \dots, C_N\},$$

where each $C_i = (\theta_i, M_{\theta_i})$ contains the sampled architecture, its instantiated model, and empty training logs. This defines the exact starting point of the evolutionary loop.

4.2.2 Batch-Level Training Protocol

Early learning dynamics provide strong predictive cues of final model accuracy (Adriaensen et al., 2023; Domhan et al., 2015; Egele et al., 2023, 2024). *ShouldYouEvenNN* exploits this property by training each neural candidate through lightweight, one-epoch cycles that capture these informative early signals without requiring convergence.

Training proceeds at the granularity of mini-batches, recording validation accuracy after fixed intervals to construct fine-grained learning curves. Each epoch uses a controlled subset of the data, which expands by a growth factor (default 1.4) in subsequent generations—maintaining a uniform rhythm while gradually increasing sample diversity.

For every batch, the framework logs training loss, validation accuracy, and elapsed time, storing them in the candidate’s ledger for later forecasting. Because all candidates share identical batch-level evaluation frequency and deterministic data order, their trajectories are directly comparable regardless of architecture or hyperparameters.

This batch-level protocol replaces the standard epoch-based evaluation used in NAS with an evidence-oriented alternative: instead of committing to full runs, it focuses computational effort on the early regions of training where most predictive information resides. These trajectories form the statistical foundation for the forecasting and decision mechanisms described next.

4.2.3 Forecasting Module

As seen in Section 2.4, forecasting strategies allow early training signals to estimate its approximated performance later on, this is key for *ShouldYouEvenNN*. The forecasting module is responsible for estimating the long-term validation accuracy of a partially trained candidate using only its early learning trajectory. It transforms short sequences of batch-level validation accuracies into predictions about future performance and associated uncertainty. This estimation forms the evidential basis for all subsequent probability and decision computations in *ShouldYouEvenNN*.

Input and Objective. Given a series of validation accuracies $y = \{y_1, y_2, \dots, y_t\}$ observed after successive mini-batches $x = \{1, 2, \dots, t\}$, the goal is to forecast the accuracy $\hat{y}_f$ expected at a future training horizon T_f (constant value set at 5). This horizon is defined as:

$$T_f = t_{\text{now}} + N \cdot t_{\text{dataset}}, \quad (4.1)$$

This formulation extrapolates the forecasting concept from mini-batch granularity to the epoch level. The term t_{dataset} denotes the number of batches in one complete epoch. Hence, the forecast horizon T_f scales proportionally with additional full passes, effectively projecting performance as if training continued for N further epochs beyond the current state. This preserves conceptual consistency between batch and epoch-level interpretations.

4.2.3.1 Shape-Aware Morphing.

Neural learning curves evolve through distinct regimes: rapid growth, plateau transition, and eventual stabilization. Relying on a single functional form (e.g., rational or logistic) often fails to capture this diversity. Following this logic, *ShouldYouEvenNN* models early validation trajectories using rational, sigmoid, and linear functions to capture the growth–plateau–stabilization pattern typical of neural training curves (White et al., 2021; Wistuba & Pedapati, 2020). To address this, *ShouldYouEvenNN* employs a *shape-aware morphing* forecaster that blends three complementary models:

$$f_{\text{morph}}(x) = w_r f_{\text{rational}}(x) + w_s f_{\text{sigmoid}}(x) + w_l f_{\text{linear}}(x),$$

where $f_{\text{rational}}(x) = \frac{ax}{b+x}$, $f_{\text{sigmoid}}(x) = \frac{L}{1+e^{-k(x-x_0)}}$, and $f_{\text{linear}}(x) = mx + c$.

The morphing weights (w_r, w_s, w_l) are not fixed parameters but adaptive coefficients derived from the local geometry of the learning curve. Specifically, they are computed from the instantaneous slope s_t and curvature c_t of the validation trajectory as follows:

$$w_r = \frac{1}{1 + e^{-5|s_t|}} \left(1 - \frac{1}{1 + e^{-5|c_t|}} \right), \quad w_s = \frac{1}{1 + e^{-3c_t}}, \quad w_l = \max(0, 1 - (w_r + w_s)).$$

Each weight encodes a distinct phase of the learning process:

- **Rational weight** (w_r). Increases with the magnitude of the slope $|s_t|$ and dominates during rapid accuracy growth, capturing the early acceleration phase of learning.
- **Sigmoid weight** (w_s). Rises with positive curvature ($c_t > 0$), emphasizing the saturation regime where progress slows and the curve bends toward a plateau.
- **Linear weight** (w_l). Becomes dominant when both slope and curvature approach zero, representing the stabilization or noise-dominated phase.

These raw weights are subsequently normalized to ensure $w_r + w_s + w_l = 1$ and softened by a temperature exponent ($w_i \leftarrow w_i^{\text{temp}}$) to guarantee smooth transitions between regimes.

Morphing Rationale. Classical learning–curve models (e.g. exponential, logistic, power-law) assume a single global trajectory observed over several full epochs. In *ShouldYouEvenNN*, how-

ever, the available evidence consists of *mini-batch-level validation points* collected under a progressively expanding data regime. During the early generations, the model sees only a small fraction of the training instances; the effective learning curve therefore spans multiple regimes in rapid succession: steep initial changes, premature saturation, and near-linear drift as the sampled instance budget grows. In this setting, no single parametric family provided consistently stable or non-overconfident extrapolations across datasets.

To avoid committing to an incorrect global form under such heterogeneous early signals, the forecasting module blends three canonical regimes commonly discussed in learning-curve literature: rapid growth (rational), saturation (sigmoid), and stabilization (linear). The resulting morphing estimator is intentionally conservative: it is not designed to reproduce the full long-term curve, but to provide a robust, low-cost indication of whether continued neural training is likely to surpass the non-neural baseline.

Weights Rationale. The weights are tied to the local shape of the curve because that is the only reliable signal available at early stages: few points, shifting data coverage, and highly variable short-term dynamics. Slope and curvature encode which regime the model is entering—acceleration, saturation, or stability—without assuming a global functional form the data have not yet settled into. This keeps the forecast stable, cheap, and conservative, which is exactly what the continuation decision requires. Together, these adaptive terms allow the forecaster to *morph* smoothly between acceleration, saturation, and stability behaviors according to the evolving shape of the validation curve. This ensures that the forecast remains realistic without relying on manually chosen functional forms.

Low points Behavior. When few validation points are available (not enough batches have been passed to make a prediction), the forecaster defaults to a conservative rule $\hat{y}_f = y_t + 0.05$, preventing over-optimistic projections from insufficient data. As more batches accumulate, the morphing weights stabilize, and the forecast converges toward the empirical trajectory.

Output. The module returns the mean forecasted validation accuracy $\hat{y}_f$, and in the broader framework, this serve as input to the scoring module, which transforms these into probabilistic evidence about whether the neural candidate is likely to surpass the non-neural baseline.

4.2.4 Probability or confidence p from the forecast.

To express the confidence that the neural model will surpass the baseline, the margin between the forecasted validation accuracy $\hat{y}_f$ and the baseline performance g is converted into a probability through a logistic mapping:

$$p = \sigma\left(\frac{\hat{y}_f - g}{t}\right),$$

where $\sigma(z) = 1/(1 + e^{-z})$ and $t > 0$ controls the smoothness of the transition, and g is the baseline metric of the non neural model. When the forecast is clearly above the baseline, p approaches 1; when it is below, p tends toward 0. This formulation provides a stable and interpretable confidence measure albeit a simple one.

4.2.5 Evolutionary Refinement

After probabilistic scoring, the candidate population undergoes a lightweight evolutionary refinement that updates the architectural pool for the next generation. This phase balances exploration and exploitation, reusing high-probability architectures while injecting controlled diversity to avoid premature convergence.

Selection. Candidates are ranked by their probability p of surpassing the non-neural baseline. A fixed top fraction ρ of the population is retained as parents for the next generation. The value of ρ is a user-defined hyperparameter (default $\rho = 0.40$) controlling the balance between exploitation and diversity.

Mutation. Each parent architecture undergoes small stochastic perturbations to structural parameters. Layer widths or dropout rates are modified by bounded multiplicative noise, and categorical attributes (e.g., activations or normalization types) may switch to a neighboring option. This promotes local exploration around successful designs without requiring full retraining.

Recombination. With lower probability, pairs of parents produce hybrid offspring whose attributes—such as depth or activation patterns—are inherited proportionally to parental scores. This operator accelerates convergence while preserving diversity.

Pruning and continuity. Candidates with persistently low probabilities or negative expected utility (see Section 4.2.6) are discarded, maintaining constant population size and focusing resources on promising regions of the search space.

Design philosophy. Unlike traditional NAS evolution schemes with large populations and heavy tuning, the refinement in *ShouldYouEvenNN* is deliberately coarse-grained and time-aware. Its role is not exhaustive optimization but the generation of informative diversity that feeds subsequent evaluation and decision phases with minimal overhead.

4.2.6 Decision Criterion Based on Expected Utility

At the end of each generation, the top candidate is evaluated by a dimensionless, cost-aware continuation score:

$$U = \underbrace{w(p \cdot (\hat{y}_f - g))}_{\text{expected benefit}} - \lambda \underbrace{C}_{\text{normalized cost}}, \quad \text{continue iff } U > 0. \quad (4.2)$$

Making U and C commensurate. All terms in equation (4.2) are constructed to be *unit-free* and bounded.

- **Benefit** is the probability-weighted accuracy margin:

$$\hat{y}_f - g \in [-1, 1], \quad p \in [0, 1], \quad p \cdot (\hat{y}_f - g) \in [-1, 1].$$

Here $\hat{y}_f \in [0, 1]$ is the forecasted validation accuracy at the task’s horizon and $g \in [0, 1]$ is the best non-neural baseline on the same split, so both share the same “units” (accuracy in $[0, 1]$). The probability p is obtained by converting forecast uncertainty into a beat-baseline probability (see 4.2.4).

- **Cost normalization.** To prevent long forecast horizons from overwhelming the expected-utility score, the remaining effort e_{rem} is mapped to a bounded interval through an exponential compression:

$$C = 1 - \exp(-e_{\text{rem}}/\tau) \in [0, 1].$$

This transformation is monotonic, smooth, and saturating: small increases in remaining effort produce almost-linear growth in cost, while very large horizons asymptotically approach 1. This avoids disproportionately penalizing candidates with long but still feasible training horizons. The parameter τ controls the rate of saturation, acting as a soft scaling factor that regulates how aggressively cost influences the decision.

- **Evidence weighting.** The weighting ramp w follows a standard design principle in learning-curve forecasting and sequential decision-making. Early validation points exhibit high variance and unstable slopes, while later points provide more reliable evidence. A smooth, monotone mapping is therefore needed to increase trust in the forecast as more batches are observed. The exponential form

$$w = w_{\min} + (1 - w_{\min}) \left(1 - e^{-e_{\text{spent}}/\kappa}\right)$$

is a conventional way to encode this behaviour: it is monotonic, saturating, and controlled by a single scale parameter κ . Small increases in e_{spent} produce only mild growth in w , keeping the utility conservative when evidence is scarce, while larger budgets yield $w \rightarrow 1$, reflecting higher confidence in the forecasted improvement.

4.2.7 Refinement Phase

Once the decision criterion signals that further neural training is justified, *ShouldYouEvenNN* enters the *refinement phase*. This stage focuses on confirming the candidate’s true potential under controlled extended training while keeping total cost bounded.

Triggering. The refinement phase is activated when the continuation condition $U > 0$ is met for the top-ranked candidate. At that moment, the evolutionary search loop halts, and the remaining global time budget is reallocated to fine-tune that candidate. If no candidate ever reaches positive utility before the budget expires, a single fallback refinement is performed on the most promising forecasted model to verify whether neural networks are genuinely uncompetitive for the given dataset.

Training Procedure. During refinement, the selected candidate resumes training under the same data preprocessing, batch order, and hyperparameters used during the search. An early-stopping mechanism monitors the validation accuracy and terminates training when one of the following conditions is satisfied:

1. the validation accuracy surpasses the non-neural baseline metric;
2. no measurable improvement occurs within a fixed patience window; or
3. the remaining time budget is exhausted.

The best validation checkpoint is automatically restored at the end of training to ensure comparability.

Outcome. The refinement yields a definitive judgment for the dataset:

- If at least one candidate surpasses the baseline, the dataset is labeled as *neural worthy*;
- otherwise, the system concludes that neural networks are not competitive under the given computational constraints.

Both successful and unsuccessful refinements are logged with their final validation and test accuracies, allowing later analysis of true positive and false negative outcomes.

Role in the Framework. The refinement phase provides empirical closure to the decision process. It validates whether the probabilistic forecast and expected-utility indicator produced a correct continuation call, grounding the theoretical decision in observed performance. Through this final step, *ShouldYouEvenNN* transforms early statistical evidence into a verifiable experimental conclusion.

Chapter 5

Empirical Evaluation

5.1 Research Questions

The empirical study addresses three complementary questions, each corresponding to one of the framework’s objectives and evaluated through the experiments described in this chapter.

RQ1: Forecast Reliability: Can early batch-level validation signals, modeled through the shape-aware morphing forecaster, reliably approximate the final validation performance of neural networks across diverse datasets?

RQ2: Decision effectiveness and efficiency: Can *ShouldYouEvenNN* distinguish when continued neural training is worth it against non neural baselines based on early evidence?

RQ3: External competitiveness: Can *ShouldYouEvenNN* recover architectures whose performance approximates that of exhaustive random searches like LCBench, while operating with drastically lower computational cost?

5.2 Experimental Setup

The empirical evaluation of *ShouldYouEvenNN* follows a standardized setup designed to isolate the behavior of its forecasting, decision, and refinement mechanisms from dataset-specific variability. All experiments were conducted using deterministic seeds and identical preprocessing, ensuring full reproducibility across runs.

5.2.1 Datasets

A total of seventy-one datasets were retrieved from the *OpenML* repository (See Appendix 8) for a complete detail of these ids and its results. These datasets span binary and multiclass classification tasks of varying dimensionality, sample size, and class balance, providing a diverse benchmark for evaluating both neural and non-neural performance. Each dataset was preprocessed through the deterministic pipeline explained in Section 5.2.2.

5.2.2 Data Preprocessing Pipeline

To ensure consistency and reproducibility across all datasets, a deterministic preprocessing pipeline was implemented in Python and PyTorch. The pipeline transforms raw data into standardized representations suitable for both neural and non-neural models.

1. **Input handling.** All datasets are loaded into a unified schema with explicit data types. Categorical features are label-encoded, and numerical features are cast to 32-bit floating point values for compatibility with PyTorch tensors.
2. **Missing values.** Missing entries are imputed according to data type: numerical attributes by mean value, categorical attributes by mode. The imputation parameters are computed exclusively from the training split and reused for validation and testing.
3. **Normalization.** Numerical features are standardized to zero mean and unit variance using statistics derived from the training subset only. This normalization is applied consistently across splits to prevent data leakage.
4. **Rare-category handling.** Categories with frequency below a threshold are mapped to a shared `__other__` token to stabilize model training and ensure consistent encoding across folds.
5. **Splitting protocol.** Each dataset is divided into training, validation, and test subsets with an 80/10/10 proportion using stratified sampling for classification tasks. The random seed for splitting is fixed globally to maintain deterministic partitions across experiments.

The preprocessing pipeline guarantees that subsequent evaluations depend solely on model behavior rather than stochastic data artifacts. By enforcing a consistent transformation across all datasets, *ShouldYouEvenNN* isolates the methodological contribution—forecasting and decision-making—from dataset-specific variance. This separation allows reliable comparison of neural and non-neural outcomes under strictly controlled input conditions.

5.2.3 Candidate Representation and Search Space

A candidate encapsulates three elements: the model instance, its architectural specification, and the record of its training and validation metrics. This structure tracks both static attributes (such as layer configuration and optimizer) and dynamic evidence collected during training (batches passed, validation accuracy, forecasted performance, and probabilistic score).

- **Architecture definition.** Each candidate corresponds to a multilayer perceptron (MLP) whose depth, width, activation functions, and regularization parameters are sampled from a bounded search space.
- **Training state.** During the batch-level loop, the candidate accumulates per-batch runtime, metrics, and derived indicators such as the forecasted accuracy or expected utility.
- **Reproducibility.** All candidates are instantiated under a fixed random context to guarantee that identical seeds produce identical networks and training behavior across runs.

Search Space Design. The search space is deliberately simple yet expressive, balancing structural diversity with computational feasibility. Table 5.1 summarizes the configuration ranges used throughout the framework.

Table 5.1: Search-space configuration for neural candidates.

Hyperparameter	Range / Options
Hidden layers	2–25
Neurons per layer	3–100
Activation functions	ReLU, LeakyReLU, Sigmoid, Tanh, ELU, GELU
Dropout	0.0–0.5 (step 0.1)
Optimizer	Adam, SGD, RMSprop, AdamW
Learning rate	10^{-4} – 10^{-1} (log-uniform)
Batch size	32–1024 (powers of two)
Weight decay	0 – 10^{-2}
Momentum (SGD only)	0.8–0.99
LR scheduler	Step, Exponential, Cosine, None
Shape pattern	Constant, Pyramid, Inverted, Hourglass, Triangular, Irregular

Shape Pattern. The *shape pattern* parameter determines how neuron counts evolve across the hidden layers, influencing representational capacity and gradient flow:

- **Constant:** uniform width across layers.
- **Pyramid:** progressively narrower layers, forming a funnel topology.
- **Inverted:** increasing width toward deeper layers.
- **Hourglass:** compression followed by expansion, similar to autoencoders.
- **Triangular:** width increases then decreases symmetrically around a midpoint.
- **Irregular:** neuron counts sampled randomly within allowed limits to increase architectural diversity.

This parametrization provides sufficient expressive power to explore a wide variety of functional topologies while keeping the computational footprint predictable. Subsequent components of *ShouldYouEvenNN* exploit this representation to train, forecast, and evolve architectures under strictly controlled budgets.

5.2.4 Baseline Models

To establish reference points for comparison, three non-neural baselines were employed:

1. **NaiveAutoML** (Mohr & Wever, 2023): a minimal yet competitive AutoML framework that evaluates classical algorithms such as Gradient Boosting, Random Forests, and Logistic Regression. For this experiment it was used without hyperparameter tuning.
2. **Hist Gradient Boosting Classifier (HGB)**: an optimized tree-based ensemble implemented in `scikit-learn`, used as a strong single-model baseline.
3. **Standard MLP**: a feedforward neural network trained conventionally to full convergence, serving as a neural reference for unrestricted training cost.

The highest validation accuracy among the non-neural baselines serves as the *goal metric* g , representing the performance threshold that *ShouldYouEvenNN* attempts to surpass.

5.2.5 Search Configuration

For each dataset, a population of $N = 30$ neural candidates was sampled from the bounded search space defined in Table 5.1. Each candidate followed the batch-level training protocol described in Section 4.2.2, using the following configuration:

- initial data usage of 10% of the training set per candidate,
- single-epoch training per generation with instance growth factor $1.4\times$,

Each generation performed forecasting, scoring, and pruning before generating a new population through mutation and crossover.

5.2.6 Evaluation Protocol

The framework’s decision process was tested under a fixed global time budget:

$$T_{\text{budget}} = \max(T_{\text{HGB}} \cdot \beta_{\text{budget}}, T_{\text{MLP}}, 60 \text{ s}),$$

where T_{HGB} is the training duration of the HGB baseline, $\beta_{\text{budget}} = 3$ a scaling factor for exploration depth, and 60 seconds ensures a minimum runtime for small datasets.

At each generation:

1. validation accuracy curves were collected at batch level;
2. forecasts and confidence intervals were computed using the shape-aware morphing model;
3. probabilities of surpassing the baseline were estimated;
4. expected-utility values were computed for the best candidate;
5. if $U > 0$, refinement was triggered under early stopping; otherwise, the search continued.

All results, including forecast values, confidence intervals, decision outcomes, and training times, were stored in structured ledgers for later aggregation and analysis.

5.2.7 Benchmark Comparison: LCBench as High-Fidelity Reference

To contextualize the external validity of *ShouldYouEvenNN* for **RQ3**, we require a reference that reflects what neural performance would look like under fully trained and economically unconstrained NAS evaluations. *LCBench* (Zimmer et al., 2020) fulfills this role.

LCBench systematically trains 2,000 randomly sampled MLP configurations on 35 OpenML datasets, each for up to 50 epochs, and records full learning curves. This constitutes a high-fidelity approximation of unguided neural exploration under substantial compute budgets, providing an empirical upper bound for neural performance when cost is not a constraint.

In contrast, *ShouldYouEvenNN* purposefully avoids such exhaustive training. It halts weak candidates after their first epoch and allocates computation only when early evidence supports continuation. Comparing SYENN’s final models to the best LCBench outcomes therefore evaluates how much neural performance can be recovered with a fraction of the compute. LCBench is not a competitor, but a high-cost reference point against which the efficiency and external validity of SYENN can be assessed.

Chapter 6

Results

This section presents the empirical results obtained from the experiments described in the previous section, organized according to the three research questions that guide this study. Each subsection addresses one question, summarizing the evidence, findings, and their interpretation in the context of *ShouldYouEvenNN*’s objectives.

6.1 RQ1 – Forecast Reliability

The first research question evaluates whether early batch-level validation signals can reliably forecast the final validation accuracy of neural networks for the later purposes of this research. To test this, 25 random MLP architectures were trained on each dataset for three full passes through the data. At each batch step, validation accuracies were logged, and the shape-aware morphing model produced forecasts of long-term performance at 50 epochs in the future together with confidence intervals. The resulting forecasts were compared against the final accuracies obtained after full early-stopping training.

Results. Across the 35 datasets tested in LCBench, the mean absolute error (MAE) between forecasted and realized validation accuracies averaged 0.1465, with a root mean squared error (RMSE) of 0.1920 and a mild negative bias of -0.0324. Forecasts were therefore slightly conservative, tending to underestimate true performance—a desirable property for a system that must avoid false optimism. The average confidence interval width was 0.26, with a coverage of 0.56, indicating that uncertainty was under-calibrated across datasets. Pearson and Spearman correla-

tions between forecasted and realized accuracies were modest ($r = \mathbf{0.20}$, $p = 0.21$), suggesting that the forecaster captures performance trends rather than precise rankings.

In the Figure 6.1, an example of the learning curves of some of the models trained according to the *ShouldYouEvenNN* strategy is observed.

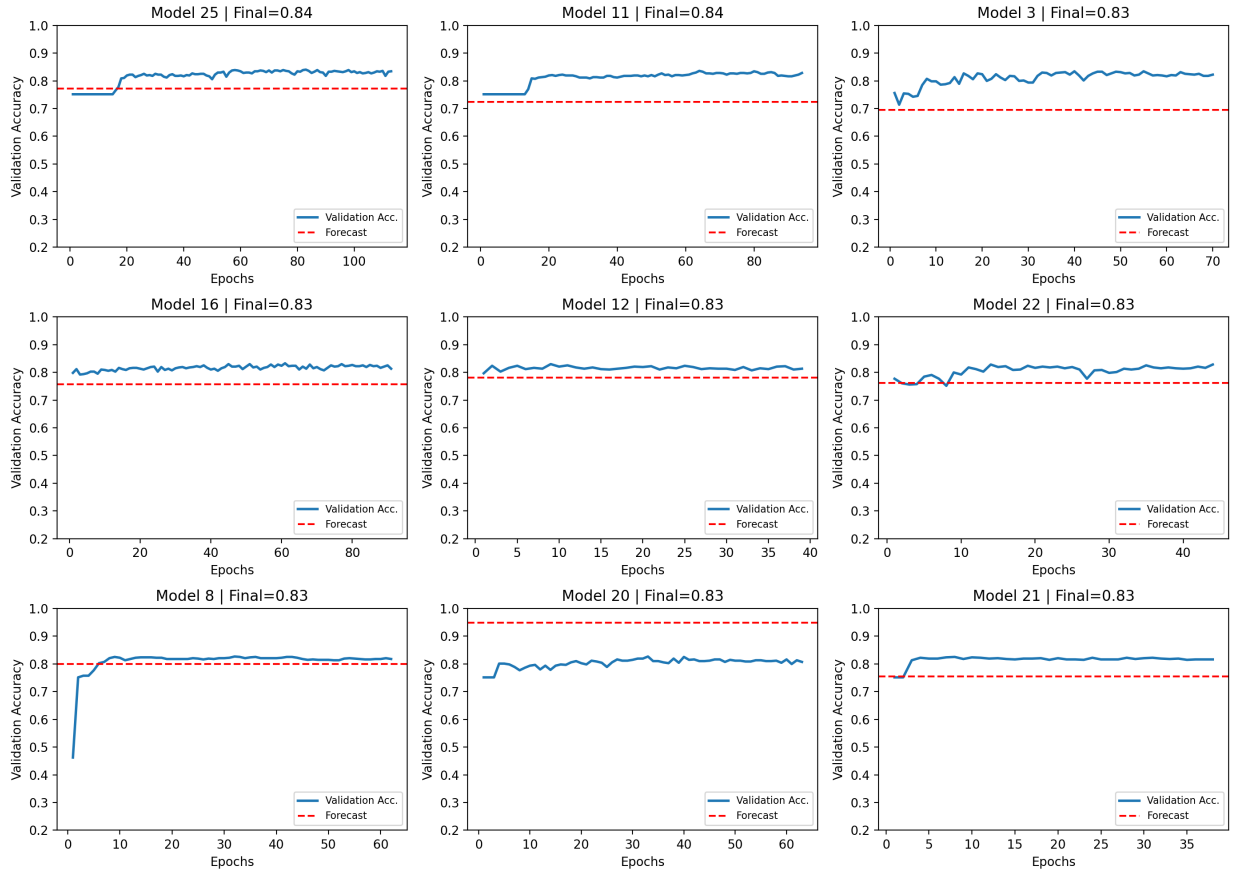
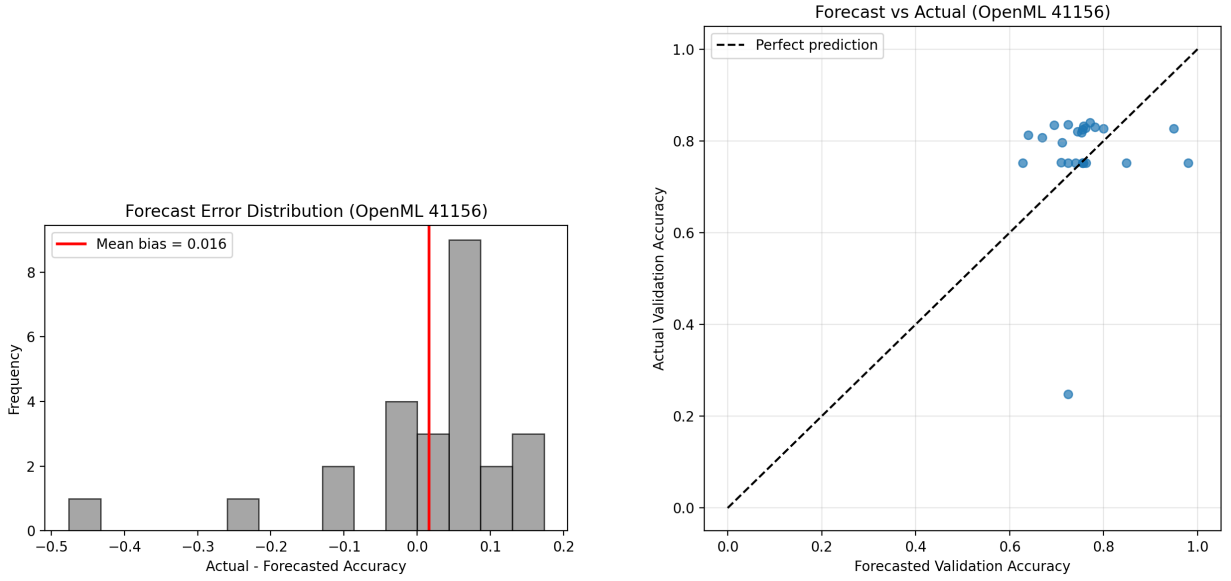


Figure 6.1: Example of learning curves for several models trained under the *ShouldYouEvenNN* strategy with their respective forecasted accuracy on the ada dataset. The trajectories illustrate the early-stage validation behavior used for forecasting.

Figure 6.2 show the error distribution with its respective MAE as well as the scatterplot with the final validation accuracy versus the forecasted one.



(a) Error distribution of forecasted validation accuracies with corresponding mean absolute error (MAE).

(b) Scatter plot comparing forecasted and observed validation accuracies.

Figure 6.2: Forecasting performance visualization on the ada dataset.

Interpretation. The results indicate that early learning curves do contain usable predictive structure, but the forecasts are far from precise. Errors and under-coverage are expected given the simplicity of the morphing model, and the correlations confirm that the forecaster captures broad trends rather than fine-grained rankings. This is sufficient for the framework’s purpose: the forecasts act as coarse signals for early continuation or rejection, not as exact estimators of final accuracy. In that limited role, even imperfect predictions provide enough directional information to support the conservative decision strategy used by *ShouldYouEvenNN*. Although the absolute correlation is low, the conservative bias ensures that *ShouldYouEvenNN* errs on the side of caution—terminating unpromising candidates early while still identifying genuinely competitive ones. Hence, the forecasting mechanism fulfills its role as a binary decision aid rather than a precise regressor, successfully addressing **RQ1**.

6.2 RQ2 – Decision Effectiveness and Efficiency

The second research question examines whether *ShouldYouEvenNN* makes accurate and efficient continuation decisions. This analysis integrates the forecasting, scoring, and decision layers described in the Methodology, evaluating whether the framework correctly identifies when neural

exploration is worthwhile under limited budgets.

Results. Across the 71 OpenML datasets (see Appendix 8 for the full list of datasets), the framework marked 38 as *neural worthy*, representing approximately 54% of all cases. The aggregated outcomes are summarized in Table 6.1.

Metric	Count
Total Datasets	71
Neural Worthy	38
Validation Surpassed	16
Test Surpassed	8

Table 6.1: Summary of decision outcomes across all evaluated datasets. Each entry aggregates the empirical results obtained after executing the complete *ShouldYouEvenNN* pipeline.

Interpretation.

- **Total Datasets** — the number of OpenML tasks used for evaluation. Each dataset underwent identical preprocessing, baseline assessment, and limited-budget neural search.
- **Neural Worthy** — datasets where the expected-utility decision criterion indicated that further neural training was justified ($U > 0$). In these cases, the framework predicted that at least one neural configuration would likely outperform the non-neural baseline.
- **Validation Surpassed** — among the neural-worthy datasets, those for which the refined candidate actually exceeded the baseline validation accuracy after extended training.
- **Test Surpassed** — "neural worthy" datasets where the advantage persisted on the held-out test set, confirming generalization beyond validation.

These metrics together quantify how frequently *ShouldYouEvenNN* correctly identified problems where neural models offer a tangible benefit. This cascade reveals the framework’s intended conservativeness: the decision mechanism filters aggressively, favoring reliability and avoiding overfitting rather than chasing marginal gains. To evaluate whether the expected-utility of the best candidate produces a meaningful separation between neural-worthy and non-neural-worthy tasks, the metric was examined across datasets of varying dimensionality. The relationship between

dataset size and the resulting utility values offers a compact view of how early evidence, cost estimates, and probabilistic forecasts interact under the framework’s continuation rule. Figure 6.3 illustrates how the expected-utility signal varies with dataset size.

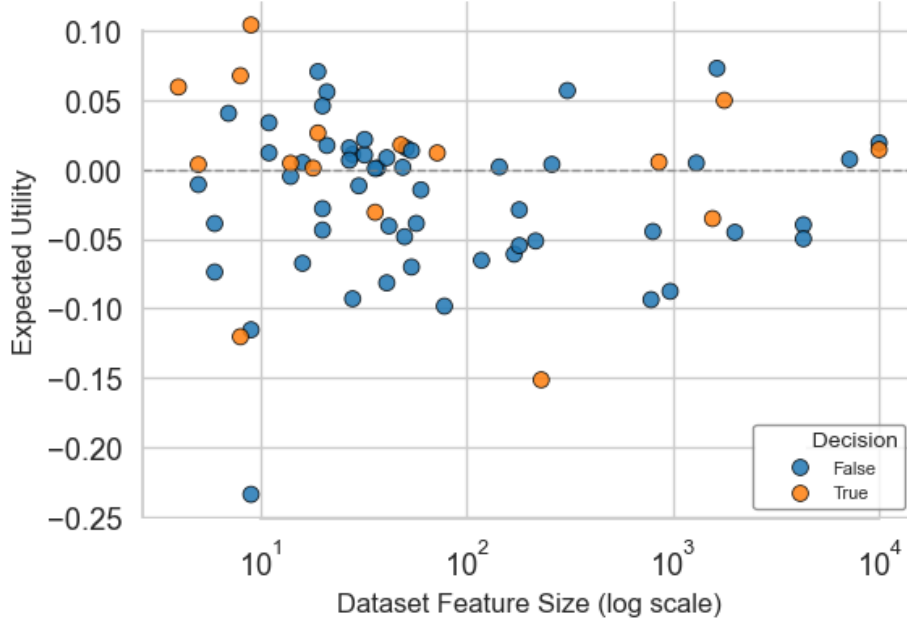


Figure 6.3: Expected-utility signal across datasets with increasing dataset size. Orange markers denote datasets deemed neural worthy (exceeded validation baseline metric); blue markers represent rejected cases.

The distribution in Figure 6.3 reveals a clear separation between positive and negative expected-utility regions. Out of 71 datasets, the sign of the expected-utility metric correctly matched the final outcome (*neural worthy* vs. rejected) in 57.7% of cases, achieving a recall of 0.75 and a precision of 0.32. This asymmetry reflects a deliberate design choice: the decision function is calibrated to favor recall over precision, ensuring that most truly neural-worthy tasks are detected even if a few false positives are tolerated. In practice, this means that the framework rarely misses datasets where neural models can meaningfully outperform strong non-neural baselines, though it occasionally allocates limited additional training to borderline cases.

Only four datasets (5.6%) corresponded to false negatives—instances where the expected-utility signal was negative yet the extended refinement phase ultimately produced a model surpassing the baseline metric. These rare reversals demonstrate the conservative reliability of the decision mechanism and validate the inclusion of a final safeguard phase to catch late-emerging improvements. Overall, the consistency between decision outcomes and the sign of expected utility supports its use as a robust scalar summary of early evidence: **when $U > 0$, continuation is**

generally justified; when $U < 0$, stopping is almost always the correct call.

As a side-note, most datasets with positive expected utility ($U > 0$) cluster below a few hundred features, suggesting that *ShouldYouEvenNN* tends to favor neural architectures when the feature space is relatively low-dimensional and structured. Conversely, for high-dimensional datasets (beyond 10^3 features), expected utility values remain largely negative, implying that the computational cost or instability outweighs potential gains in these scenarios.

Interpretation. The decision mechanism behaves as intended: conservative, evidence-driven, and explicitly cost-aware. Across datasets, the expected-utility signal correctly distinguishes between cases where continued neural training is worthwhile and those where it is not, with a deliberate bias toward recall. This bias ensures that genuinely neural-worthy datasets are rarely missed, even if the framework occasionally allocates limited additional computation to borderline cases. The moderate rate of false positives and the very small number of false negatives reflect this design choice. In practice, the expected-utility criterion functions as a reliable early classifier of neural worthiness under constrained budgets, preventing wasted computation while preserving opportunities for meaningful neural improvement. Consequently, **RQ2** is supported: *ShouldYouEvenNN* provides stable, efficient continuation decisions aligned with its methodological goals.

6.3 RQ3 – External Competitiveness

To address **RQ3**, the goal is not to compare *ShouldYouEvenNN* against LCBench as if both were competing NAS algorithms. Instead, LCBench is used as a *high-fidelity reference* that approximates the outcomes of an unrestricted random neural search. Because LCBench provides thousands of fully trained neural configurations per dataset, it effectively represents the performance frontier obtainable under unguided neural exploration.

In this study, LCBench therefore serves as a *counterfactual ground truth* for what a full neural search would yield if every architecture were trained to convergence. The evaluation examines whether the lightweight, evidence-driven early-rejection strategy of *ShouldYouEvenNN* can recover neural models whose performance approaches this frontier, but at a fraction of the computational cost. This framing allows us to assess the *external validity* of the continuation decisions

produced by the proposed framework.

Results. As shown in Figure 6.4, this distinction translates into a median runtime reduction of approximately $\times 56$, demonstrating the magnitude of computational savings achievable when early forecasts replace random exhaustive training.

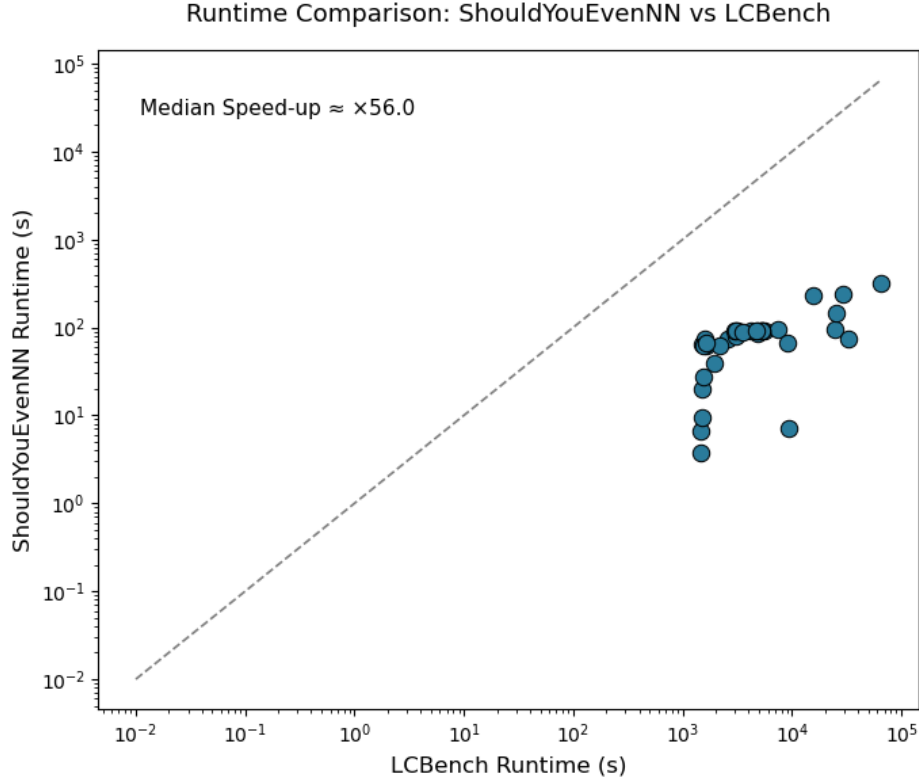


Figure 6.4: Runtime comparison between *ShouldYouEvenNN* and the Learning Curve Benchmark (LCBench) across shared datasets. Each point represents one dataset, with both axes in logarithmic scale. The dashed diagonal denotes equal runtime: points below it indicate faster completion by *ShouldYouEvenNN*.

Figure 6.5 presents a comparative analysis between the validation accuracies achieved by *ShouldYouEvenNN* and those reported by LCBench.

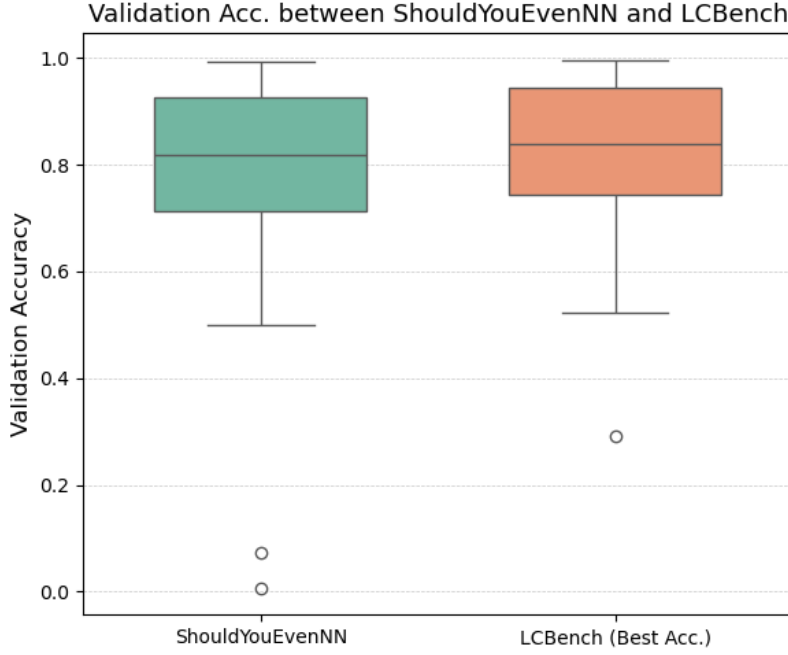


Figure 6.5: Distribution of validation accuracies obtained by *ShouldYouEvenNN* and LCBench across datasets. For LCBench, values correspond to the best validation accuracy reported by the benchmark tool for each dataset, rather than the full set of configurations included in its internal benchmark. This comparison highlights the efficiency of *ShouldYouEvenNN*, which approaches the upper performance range of LCBench despite operating under strict computational budgets.

The visual comparison indicates that *ShouldYouEvenNN* achieves median performance levels close to those of the best LCBench references. Despite operating with a minimal fraction of the computational cost (as seen in 6.4), *ShouldYouEvenNN* consistently reached accuracies within 5–6 percentage points of the fully trained NAS models. This demonstrates that early forecasting and evidence-driven rejection can recover architectures whose performance closely approximates that of exhaustive NAS benchmarks, while requiring only a fraction of the computational effort.

A contingency analysis of surpassing outcomes is shown in Table 6.2.

Table 6.2: Cross-tabulation of datasets where *ShouldYouEvenNN* and LCBench surpassed their respective baselines.

	LCBench Surpassed	ShouldYouEvenNN Surpassed	Count
1	True	True	4
3	True	False	2
2	False	True	2
0	False	False	27

The contingency analysis reveals four distinct regions of agreement between *ShouldYouEvenNN* and LCBench. In four datasets, both frameworks surpassed their respective baselines, indicating

strong overlap in identifying genuinely “neural-worthy” tasks. These cases demonstrate that even under tight budget constraints, *ShouldYouEvenNN* converges on similar success conditions as a fully exhaustive NAS procedure.

In two datasets, only LCBench surpassed its baseline while *ShouldYouEvenNN* did not, suggesting that the early-stopping and cost-aware continuation criteria occasionally terminate runs that could have yielded marginal gains with extended training. Conversely, in two datasets, *ShouldYouEvenNN* exceeded the baseline whereas LCBench did not, an inversion that highlights the framework’s selective efficiency and ability to uncover strong configurations without full exploration. The majority of cases (27) correspond to mutual non-surpassing outcomes, reflecting datasets where neither method found neural architectures competitive with non-neural baselines.

Overall, this joint distribution underscores the framework’s intended conservativeness: *ShouldYouEvenNN* rarely pursues unpromising neural exploration and still manages to recover most of the successful cases identified by exhaustive NAS. The overlap between the two “True–True” cells and the scarcity of “True–False” cases validate the reliability of its decision rule—suggesting that when *ShouldYouEvenNN* deems neural training worthwhile, it almost always aligns with high-fidelity NAS results.

Interpretation. Despite operating under a strict computational budget, *ShouldYouEvenNN* consistently approaches the performance frontier represented by exhaustive NAS. In the comparison with LCBench, the framework recovers architectures typically within 5–6 percentage points of the benchmark’s fully trained models, yet at a fraction of the runtime. This proximity indicates that early, uncertainty-aware continuation decisions can reliably capture much of the neural performance frontier without resorting to large-scale random exploration. The findings confirm that a decision-oriented NAS strategy can achieve competitive accuracy while preserving high efficiency and full reproducibility, thereby addressing **RQ3**.

6.4 Summary

Collectively, the results across RQ1–RQ3 show that:

- early forecasting provides sufficiently reliable evidence for continuation decisions;

- the decision rule accurately balances benefit and cost, preventing unnecessary computation; and
- the neural architectures selected by the framework achieve performance close to that observed in high-fidelity neural benchmarks, despite being obtained under strict computational budgets.

These findings confirm that *ShouldYouEvenNN* meets its intended objectives of reliability, efficiency, and competitive performance under constrained resources.

Chapter 7

Conclusion

This final chapter synthesizes the conceptual and empirical contributions of the work, highlighting how *ShouldYouEvenNN* reframes Neural Architecture Search (NAS) as a binary decision problem.

The *ShouldYouEvenNN* framework demonstrates that early-training information, when properly modeled and integrated into a probabilistic decision process, can substantially reduce the computational burden of NAS without compromising predictive reliability. By treating the continuation of neural training as a decision under uncertainty rather than a fixed commitment, the framework introduces a principled mechanism for determining when neural exploration is justified relative to strong non-neural baselines.

This study redefines the purpose of NAS. Rather than viewing architecture discovery as a race to find the highest-accuracy network, *ShouldYouEvenNN* interprets it as a problem of rational evidence allocation, where limited computational resources are invested only when early learning signals indicate a positive expected return. In this sense, NAS becomes not merely an optimization procedure, but an adaptive inference system guided by the probabilistic value of continuing to learn.

Beyond its empirical findings, this work represents a shift in perspective for automated machine learning: one that dares to ask, *Should you even neural network?*. It bridges the efficiency discipline of AutoML with the exploratory nature of NAS, showing that progress in machine learning is not only about building larger or more complex models, but also about knowing when *not* to. By turning early evidence into judgment, *ShouldYouEvenNN* transforms model discovery from blind optimization into rational inference, demonstrating that the future of NAS lies in selective,

evidence-based continuation rather than exhaustive computation.

Limitations and Future Work

While the findings demonstrate the viability of early-evidence decision making, several limitations remain. First, the framework is tailored to tabular classification and shallow multilayer perceptrons; its applicability to convolutional, recurrent, or transformer-based architectures is untested. Second, the hyperparameters used were fixed and manually selected (see Appendix 8.7), which limits adaptation to dataset-specific characteristics and require further calibration. Third, datasets with delayed or highly noisy learning curves can weaken forecast reliability, occasionally leading to premature rejection of potentially competitive models.

Future work should explore meta-learning mechanisms to automatically calibrate the utility parameters based on dataset meta-features, extend the forecasting and decision model to more complex neural families, and integrate the approach into end-to-end AutoML systems. Such pipelines would not only train neural networks efficiently, but also determine—early and with statistical justification—when neural training is unnecessary altogether.

Bibliography

- Aach, M., Inanc, E., Sarma, R., Riedel, M., & Lintermann, A. (2023). Optimal Resource Allocation for Early Stopping-based Neural Architecture Search Methods.
- Adriaensen, S., Rakotoarison, H., Müller, S., & Hutter, F. (2023). Efficient Bayesian Learning Curve Extrapolation using Prior-Data Fitted Networks. <https://arxiv.org/pdf/2310.20447>
- Baratchi, M., Wang, C., Limmer, S., van Rijn, J. N., Hoos, H., Bäck, T., & Olhofer, M. (2024). Automated machine learning: past, present and future. *Artificial Intelligence Review*, 57(5), 122. <https://doi.org/10.1007/s10462-024-10726-1>
- Borisov, V., Leemann, T., Seßler, K., Haug, J., Pawelczyk, M., & Kasneci, G. (2022). Deep Neural Networks and Tabular Data: A Survey. *IEEE Transactions on Neural Networks and Learning Systems*, 35(6), 7499–7519. <https://doi.org/10.1109/TNNLS.2022.3229161>
- Domhan, T., Springenberg, J. T., & Hutter, F. (2015). Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves. *Proceedings of the 24th International Conference on Artificial Intelligence*, 3460–3468.
- Egele, R., Guyon, I., Sun, Y., & Balaprakash, P. (2023). Is One Epoch All You Need For Multi-Fidelity Hyperparameter Optimization? *ESANN 2023 proceedings*, 555–560. <https://doi.org/10.14428/esann/2023.ES2023-84>
- Egele, R., Mohr, F., Viering, T., & Balaprakash, P. (2024). The unreasonable effectiveness of early discarding after one epoch in neural network hyperparameter optimization. *Neurocomputing*, 597, 127964. <https://doi.org/10.1016/J.NEUCOM.2024.127964>
- Elsken, T., Metzen, J. H., & Hutter, F. (2019). *Neural Architecture Search: A Survey* (tech. rep.). <http://jmlr.org/papers/v20/18-598.html>.

- Garcia-Garcia, C., Derbel, B., Morales-Reyes, A., & Escalante, H. (2025). *Speeding up the Multi-objective NAS Through Incremental Learning* (Vol. 15247 LNAI). https://doi.org/10.1007/978-3-031-75543-9_1
- Gardner, J., Popović, Z., Schmidt, L., & Allen, P. G. (2022). Subgroup Robustness Grows On Trees: An Empirical Baseline Investigation. <https://arxiv.org/pdf/2211.12703>
- Gorishniy, Y., Rubachev, I., Khrulkov, V., & Babenko, A. (2021). Revisiting Deep Learning Models for Tabular Data. *Advances in Neural Information Processing Systems*, 23, 18932–18943. <https://arxiv.org/pdf/2106.11959>
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on tabular data? <https://arxiv.org/pdf/2207.08815>
- Guan, J., Zhang, F., Liu, J., Du, X., Shen, X., & Shen, X. A. (2025). A Systematic Study on Early Stopping Metrics in HPO and the Implications of Uncertainty. 18(6), 2150–8097. <https://doi.org/10.14778/3725688.3725689>
- Heckerman, D., Horvitz, E. J., & Middleton, B. (2013). An Approximate Nonmyopic Computation for Value of Information. <https://arxiv.org/pdf/1303.5720>
- Jin, H., Song, Q., & Hu, X. (2019). Auto-Keras: An Efficient Neural Architecture Search System. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1946–1956. <https://doi.org/10.1145/3292500.3330648>
- Jurado, I. C., & Vanschoren, J. (2022). Multi-fidelity optimization method with asynchronous generalized island model for AutoML. *GECCO 2022 Companion - Proceedings of the 2022 Genetic and Evolutionary Computation Conference*, 220–223. <https://doi.org/10.1145/3520304.3528917>
- Lin, Y., Endo, Y., Lee, J., & Kamijo, S. (2024). Bandit-NAS: Bandit sampling and training method for Neural Architecture Search. *Neurocomputing*, 597, 127684. <https://doi.org/10.1016/J.NEUCOM.2024.127684>
- Liu, X., Tian, J., Duan, P., Yu, Q., Wang, G., & Wang, Y. (2024). GrMoNAS: A granularity-based multi-objective NAS framework for efficient medical diagnosis. *Computers in Biology and Medicine*, 171, 108118. <https://doi.org/10.1016/J.COMPBIOMED.2024.108118>

- Lu, S., Hu, Y., Yang, L., Mei, J., Sun, Z., Tan, J., & Song, C. (2024). PHD-NAS: Preserving helpful data to promote Neural Architecture Search. *Neurocomputing*, 587. <https://doi.org/10.1016/j.neucom.2024.127646>
- McElfresh, D., Khandagale, S., Valverde, J., Prasad, V. C., Ramakrishnan, G., Goldblum, M., & White, C. (2023). When Do Neural Nets Outperform Boosted Trees on Tabular Data? *Advances in Neural Information Processing Systems*, 36. <https://arxiv.org/pdf/2305.02997>
- Mebirouk, N., Amrane, M., Messast, S., & Mazouzi, S. (2024). Enhanced analysis of landslide susceptibility mapping in the proximity of main roads in the province of Skikda, Algeria: using NAS for efficient performance and faster processing. *Modeling Earth Systems and Environment*, 10(5), 6449–6474. <https://doi.org/10.1007/s40808-024-02129-6>
- Mendoza, H., Klein, A., Feurer, M., Springenberg, J. T., & Hutter, F. (2016, November). Towards Automatically-Tuned Neural Networks. In F. Hutter, L. Kotthoff, & J. Vanschoren (Eds.), *Proceedings of the workshop on automatic machine learning* (pp. 58–65, Vol. 64). PMLR. https://proceedings.mlr.press/v64/mendoza_towards_2016.html
- Mohr, F., & van Rijn, J. N. (2022). Learning Curves for Decision Making in Supervised Machine Learning – A Survey. <http://arxiv.org/abs/2201.12150>
- Mohr, F., & van Rijn, J. N. (2023). Fast and Informative Model Selection Using Learning Curve Cross-Validation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(8), 9669–9680. <https://doi.org/10.1109/TPAMI.2023.3251957>
- Mohr, F., & Wever, M. (2023). Naive automated machine learning. *Machine Learning*, 112(4), 1131–1170. <https://doi.org/10.1007/s10994-022-06200-0>
- Nandal, P., Mann, P., Bohra, N., Aldehim, G., Elnour, A. A. H., & Allafi, R. (2025). Tropical cyclone intensity estimation based on YOLO-NAS using satellite images in real time. *Alexandria Engineering Journal*, 113, 227–241. <https://doi.org/10.1016/j.aej.2024.10.072>
- Nguyen, D. A., Kononova, A. V., Menzel, S., Sendhoff, B., & Back, T. (2022). An Efficient Con- testing Procedure for AutoML Optimization. *IEEE Access*, 10, 75754–75771. <https://doi.org/10.1109/ACCESS.2022.3192036>
- Patel, S., Patel, D., Patel, A., Nanavati, J., Patel, U., Faldu, A., & Shingala, A. (2024). Accurate and Efficient Real-Time Crop and Weed Identification Using YOLO-NAS: A Neural Architecture Search-Optimized Object Detection Model. *International Journal of Engineer-*

- ing Trends and Technology*, 72(10), 35–45. <https://doi.org/10.14445/22315381/IJETT-V72I10P104>
- Ren, P., Xiao, Y., Chang, X., Huang, P.-y., Li, Z., Chen, X., & Wang, X. (2022). A Comprehensive Survey of Neural Architecture Search. *ACM Computing Surveys*, 54(4), 1–34. <https://doi.org/10.1145/3447582>
- Ru, B., Lyle, C., Schut, L., Fil, M., Van Der Wilk, M., & Gal, Y. (n.d.). Speedy Performance Estimation for Neural Architecture Search. <https://github.com/rubinxin/TSE>.
- Sakamoto, K., Ishibashi, H., Sato, R., Shirakawa, S., Akimoto, Y., & Hino, H. (2023). ATNAS: Automatic Termination for Neural Architecture Search. *Neural Networks*, 166, 446–458. <https://doi.org/10.1016/J.NEUNET.2023.07.011>
- Salehin, I., Islam, M. S., Saha, P., Noman, S., Tuni, A., Hasan, M. M., & Baten, M. A. (2024). AutoML: A systematic review on automated machine learning with neural architecture search. *Journal of Information and Intelligence*, 2(1), 52–81. <https://doi.org/10.1016/j.jiixd.2023.10.002>
- Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90. <https://doi.org/10.1016/J.INFFUS.2021.11.011>
- Solís-Martín, D., Galán-Páez, J., & Borrego-Díaz, J. (2025). A Model for Learning-Curve Estimation in Efficient Neural Architecture Search and Its Application in Predictive Health Maintenance. *Mathematics 2025, Vol. 13, Page 555, 13(4)*, 555. <https://doi.org/10.3390/MATH13040555>
- Swersky, K., Snoek, J., & Adams, R. P. (2014). Freeze-Thaw Bayesian Optimization. <https://arxiv.org/pdf/1406.3896>
- Thornton, C., Hutter, F., Hoos, H. H., & Leyton-Brown, K. (2013). Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms. <https://arxiv.org/abs/1208.3719>
- van Hoof, J., & Vanschoren, J. (2021). Hyperboost: Hyperparameter Optimization by Gradient Boosting surrogate models. <https://arxiv.org/pdf/2101.02289>
- Wang, W., & Li, H. (2025). NAS FD Lung: A novel lung assist diagnostic system based on neural architecture search. *Biomedical Signal Processing and Control*, 100. <https://doi.org/10.1016/j.bspc.2024.107022>

- Wang, Z., Gehring, C., Kohli, P., & Jegelka, S. (2017). Ensemble Bayesian Optimization. *International Conference on Artificial Intelligence and Statistics, AISTATS 2018*, 745–754. <https://arxiv.org/pdf/1706.01445>
- Wei, C., Niu, C., Tang, Y., Wang, Y., Hu, H., & Liang, J. (2020). NPENAS: Neural Predictor Guided Evolution for Neural Architecture Search. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11), 8441–8455. <https://doi.org/10.1109/TNNLS.2022.3151160>
- Wen, W., Liu, H., Chen, Y., Li, H., Bender, G., & Kindermans, P. J. (2019). Neural Predictor for Neural Architecture Search. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 12374 LNCS, 660–676. https://doi.org/10.1007/978-3-030-58526-6{_}39
- White, C., Zela, A., Ru, B., Liu, Y., & Hutter, F. (2021). How Powerful are Performance Predictors in Neural Architecture Search? *Advances in Neural Information Processing Systems*, 34, 28454–28469. <https://arxiv.org/pdf/2104.01177>
- Wistuba, M., & Pedapati, T. (2020). Learning to Rank Learning Curves.
- Wu, J., Cao, M., Shan, L., & Yang, Q. (2020). Higher Performance for AutoML: The Benefit of Various Ensemble Bayesian Optimization Strategy. *34th Conference on Neural Information Processing Systems*. <https://valohaichirpprod.blob.core.windows.net/papers/duxiaoman.pdf>
- Yan, S., White, C., Savani, Y., & Hutter, F. (2021). NAS-Bench-x11 and the Power of Learning Curves. *Advances in Neural Information Processing Systems*, 27, 22534–22549. <https://arxiv.org/pdf/2111.03602>
- Zheng, X., Ji, R., Chen, Y., Wang, Q., Zhang, B., Chen, J., Ye, Q., Huang, F., & Tian, Y. (2021). MIGO-NAS: Towards Fast and Generalizable Neural Architecture Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(9), 2936–2952. <https://doi.org/10.1109/TPAMI.2021.3065138>
- Zimmer, L., Lindauer, M., & Hutter, F. (2020). Auto-PyTorch Tabular: Multi-Fidelity MetaLearning for Efficient and Robust AutoDL. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(9), 3079–3090. <https://doi.org/10.1109/TPAMI.2021.3067763>

Zoph, B., & Le, Q. V. (2016). Neural Architecture Search with Reinforcement Learning. *5th International Conference on Learning Representations, ICLR 2017 - Conference Track Proceedings*. <https://arxiv.org/abs/1611.01578v2>

Chapter 8

Appendix

8.1 Appendix A: Algorithm A.1 – Detailed Operational Flow of ShouldYouEvenNN.

Algorithm 2 Overall Operational Flow of *ShouldYouEvenNN*

Require: Training data $(X_{\text{train}}, y_{\text{train}})$, validation data $(X_{\text{val}}, y_{\text{val}})$, non-neural baseline accuracy g

Ensure: Final decision on whether neural training is worthwhile

- 1: **Initialize:** Load dataset, baselines, and define search space parameters and population size.
 - 2: **Phase 1: Candidate Training**
 - 3: Train an initial population of neural architectures under limited batch budgets.
 - 4: Log validation accuracies y_t after each batch index t .
 - 5: **Phase 2: Forecasting**
 - 6: For each candidate, fit a morphing forecast model f_{morph} to predict future validation accuracy.
 - 7: Estimate probability of improvement p relative to baseline g .
 - 8: **Phase 3: Decision and Refinement**
 - 9: Compute decision score $U = p \cdot \Delta - \lambda C$ for the best candidate.
 - 10: **if** $U > 0$ **then**
 - 11: Trigger refinement phase (early-stopping training) on the top-ranked candidate.
 - 12: If this candidate fails to surpass baseline g , proceed sequentially with the next best candidates while time budget remains.
 - 13: **else**
 - 14: No candidate reached positive score; perform fallback refinement on the best forecasted candidate.
 - 15: **end if**
 - 16: **Phase 4: Final Validation**
 - 17: Continue training each selected candidate using early stopping until validation accuracy stabilizes or time expires.
 - 18: **if** any refined candidate surpasses baseline g **then**
 - 19: Mark dataset as *neural worthy* and record best validation accuracy.
 - 20: **else**
 - 21: Conclude that neural networks are not competitive under the given budget.
 - 22: **end if**
 - 23: **Return:** Final decision, best validation accuracy, best model, and total elapsed time.
-

8.2 Appendix B: Performance across datasets

Table 8.1: Condensed ShouldYouEvenNN [SYENN] Experiment Results per Dataset

data_id	naml_best_validation_metric	naml_test_accuracy	syenn_decision	syenn_val_acc	syenn_test_acc	syenn_surpassed_val_baseline	syenn_surpassed_test_baseline	syenn_time_used_s
3	0.9941	0.9969	False	0.9941	0.9891	True	False	61.3809
12	0.9850	0.9725	False	0.9750	0.9550	False	False	61.4269
23	0.5619	0.5153	True	0.6017	0.4881	True	False	9.0699
31	0.7575	0.7300	True	0.7562	0.7100	False	False	19.7634
54	0.8515	0.8412	True	0.8824	0.7529	True	False	9.3394
181	0.6050	0.6364	True	0.6345	0.5690	True	False	6.6603
188	0.6661	0.6622	True	0.6780	0.6554	True	False	5.2224
1049	0.9179	0.8836	True	0.9060	0.9110	False	True	19.5543
1067	0.8870	0.8412	True	0.8787	0.8318	False	False	27.6183
1111	0.9822	0.9822	False	0.9822	0.9822	True	True	86.5667
1169	0.6579	0.6607	True	0.6256	0.6293	False	False	93.6522
1457	0.8133	0.8600	True	0.5708	0.4933	False	False	177.8856
1461	0.9133	0.9060	True	0.8831	0.8830	False	False	80.0106
1464	0.7817	0.7733	True	0.8083	0.7867	True	True	3.7188
1468	0.9353	0.9398	True	0.9769	0.9167	True	False	7.0944
1475	0.6047	0.6258	True	0.5444	0.5368	False	False	68.5845
1486	0.9699	0.9697	False	0.9476	0.9504	False	False	90.4251
1487	0.9491	0.9310	True	0.9581	0.9231	True	False	19.1388
1489	0.9210	0.8918	False	0.8960	0.8788	False	False	73.6999
1494	0.8911	0.8720	True	0.8876	0.8768	False	True	20.5712
1515	0.9087	0.9130	True	0.8261	0.7652	False	False	25.4087
1590	0.8734	0.8774	False	0.8237	0.8310	False	False	90.7140

Continued on next page

Table 8.1: Condensed ShouldYouEvenNN Experiment Results per Dataset

data_id	naml_best_validation_metric	naml_test_accuracy	syenn_decision	syenn_val_acc	syenn_test_acc	syenn_surpassed_val_baseline	syenn_surpassed_test_baseline	syenn_time_used_s
1596	0.9507	0.9551	False	0.7219	0.7233	False	False	237.2460
4134	0.7939	0.8362	True	0.8050	0.7963	True	False	36.5215
4135	0.9502	0.9469	False	0.9420	0.9422	False	False	73.4512
4534	0.9755	0.9724	False	0.9700	0.9679	False	False	71.5146
4538	0.6709	0.6911	True	0.5652	0.5727	False	False	69.8245
4541	0.6131	0.6152	True	0.5851	0.5825	False	False	90.6034
23512	0.7197	0.7260	True	0.7161	0.7178	False	False	90.3425
23517	0.5233	0.5205	True	0.5059	0.5061	False	False	90.4725
40498	0.6718	0.6827	True	0.5778	0.6153	False	False	62.7789
40668	0.8272	0.8260	False	0.6609	0.6563	False	False	90.3981
40670	0.9582	0.9483	False	0.9529	0.9310	False	False	62.0052
40685	0.9998	0.9995	False	0.7860	0.7859	False	False	88.2175
40701	0.9604	0.9630	False	0.8588	0.8590	False	False	61.7084
40900	0.9967	0.9922	True	0.9951	0.9902	False	False	60.9084
40975	0.9949	0.9971	False	0.9783	0.9682	False	False	64.0157
40978	0.9790	0.9802	False	0.9886	0.9756	True	False	63.4900
40981	0.9081	0.8043	True	0.9189	0.7464	True	False	6.5967
40982	0.7878	0.7738	True	0.7685	0.7147	False	False	54.8814
40983	0.9819	0.9845	True	0.9935	0.9897	True	True	28.4800
40984	0.9395	0.9481	False	0.9216	0.9264	False	False	62.0967
40996	0.8971	0.9024	False	0.7740	0.7631	False	False	227.8870
41027	0.9007	0.8964	False	0.8190	0.8101	False	False	90.5901
41138	0.9938	0.9934	False	0.9819	0.9819	False	False	90.7054
41142	0.7489	0.7565	True	0.7428	0.7223	False	False	73.4481

Continued on next page

Table 8.1: Condensed ShouldYouEvenNN Experiment Results per Dataset

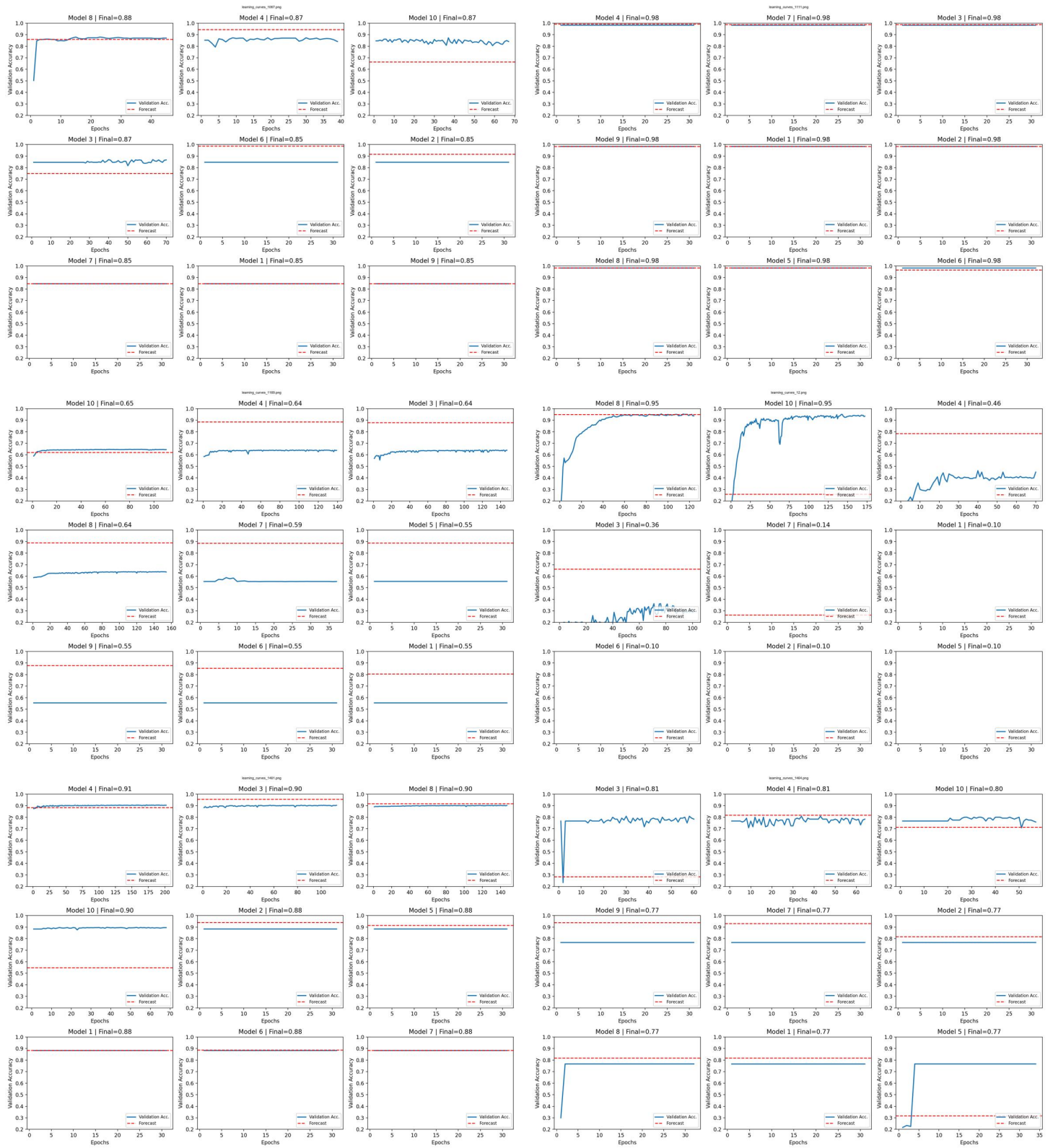
data_id	naml_best_validation_metric	naml_test_accuracy	syenn_decision	syenn_val_acc	syenn_test_acc	syenn_surpassed_val_baseline	syenn_surpassed_test_baseline	syenn_time_used_s
41143	0.8187	0.8291	True	0.8096	0.7672	False	False	39.6664
41144	0.8330	0.8551	True	0.6322	0.6178	False	False	61.4547
41145	0.7596	0.7686	True	0.7342	0.6847	False	False	52.0210
41146	0.9524	0.9424	False	0.8951	0.8985	False	False	65.6602
41147	0.6868	0.6853	False	0.5000	0.5000	False	False	143.3326
41150	0.9424	0.9424	False	0.9296	0.9255	False	False	95.5820
41156	0.8444	0.8554	True	0.8464	0.8470	True	False	59.7082
41157	0.8000	0.8000	True	0.8750	0.8000	True	True	3.0432
41158	0.9525	0.9398	False	0.9366	0.9144	False	False	64.2103
41159	0.8228	0.8320	False	0.6012	0.6000	False	False	119.5335
41161	0.9969	0.9972	False	0.9903	0.9922	False	False	144.7661
41162	0.8879	0.8858	True	0.8770	0.8770	False	False	90.5911
41163	0.9842	0.9860	False	0.9419	0.9415	False	False	160.3988
41164	0.7183	0.7130	False	0.6472	0.6292	False	False	65.7742
41165	0.5092	0.5320	True	0.3725	0.3380	False	False	1023.5956
41166	0.6788	0.6911	False	0.5175	0.5111	False	False	90.8297
41167	0.6986	0.5241	False	0.0059	0.0059	False	False	320.6525
41168	0.7105	0.7195	True	0.7082	0.7053	False	False	90.6231
41169	0.3751	0.3723	True	0.0729	0.0713	False	False	93.1162
42732	0.8784	0.8784	False	0.8784	0.8784	True	True	142.6315
42733	0.8360	0.8358	True	0.8317	0.8315	False	False	77.5396
42734	0.7500	0.7491	True	0.7158	0.7157	False	False	90.7542
42742	0.9636	0.9636	False	0.9636	0.9636	False	True	117.7782
42746	1.0000	1.0000	False	0.5732	0.5732	False	False	207.5776

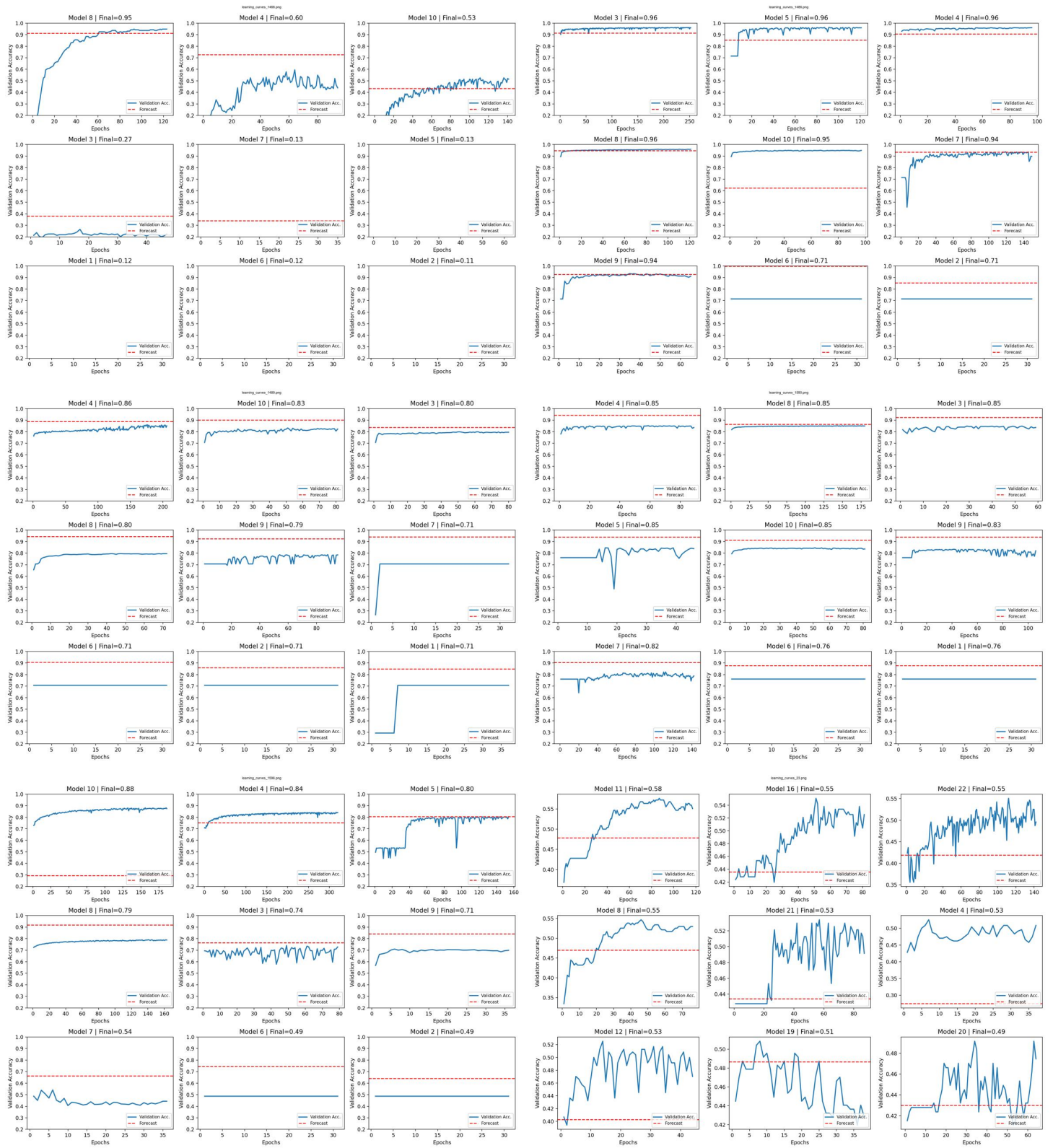
Continued on next page

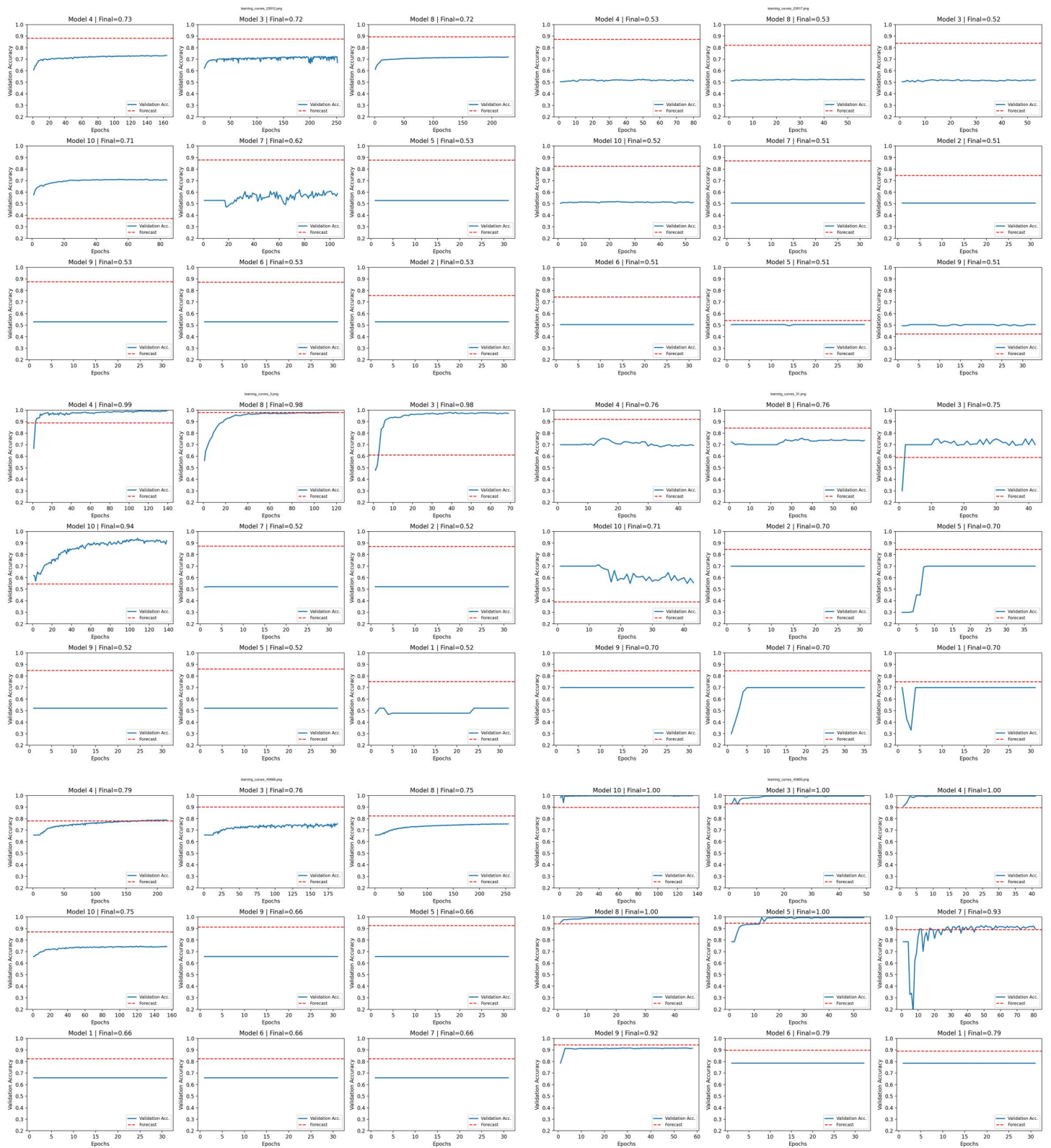
Table 8.1: Condensed ShouldYouEvenNN Experiment Results per Dataset

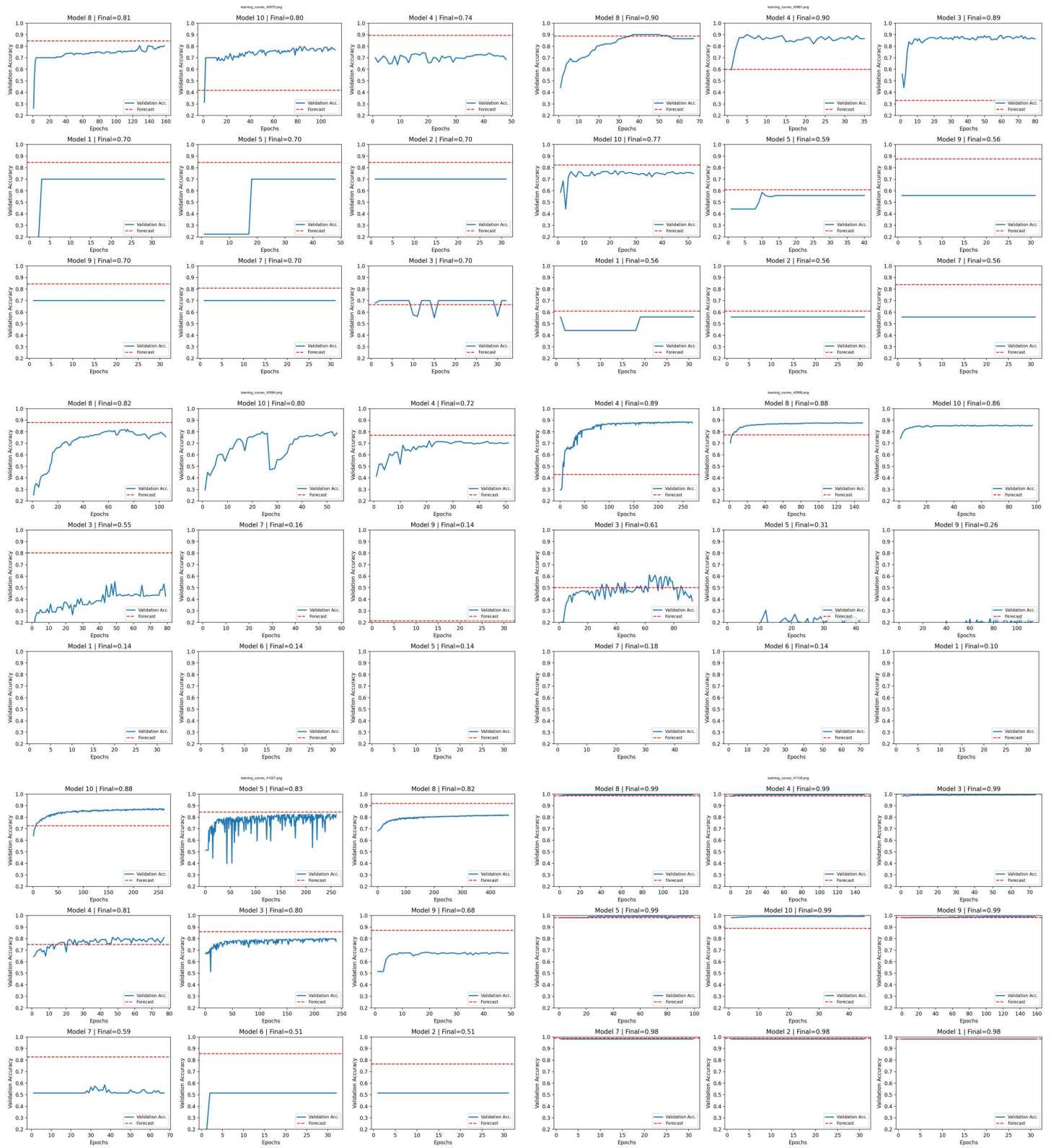
data_id	naml_best_validation_metric	naml_test_accuracy	syenn_decision	syenn_val_acc	syenn_test_acc	syenn_surpassed_val_baseline	syenn_surpassed_test_baseline	syenn_time_used_s
42769	0.7344	0.7355	False	0.5299	0.5299	False	False	208.1121

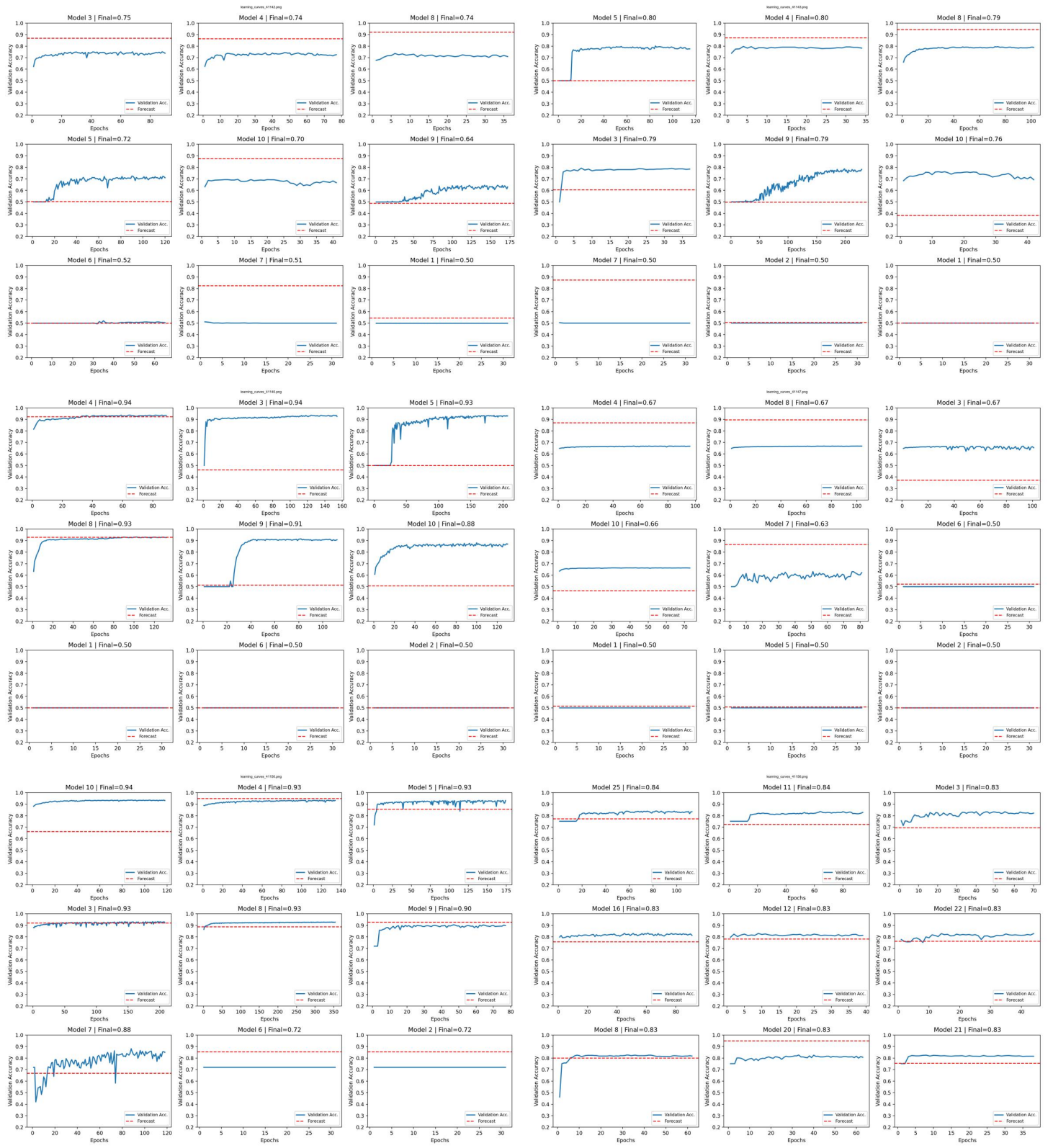
8.3 Appendix C: Learning Curves vs Forecast

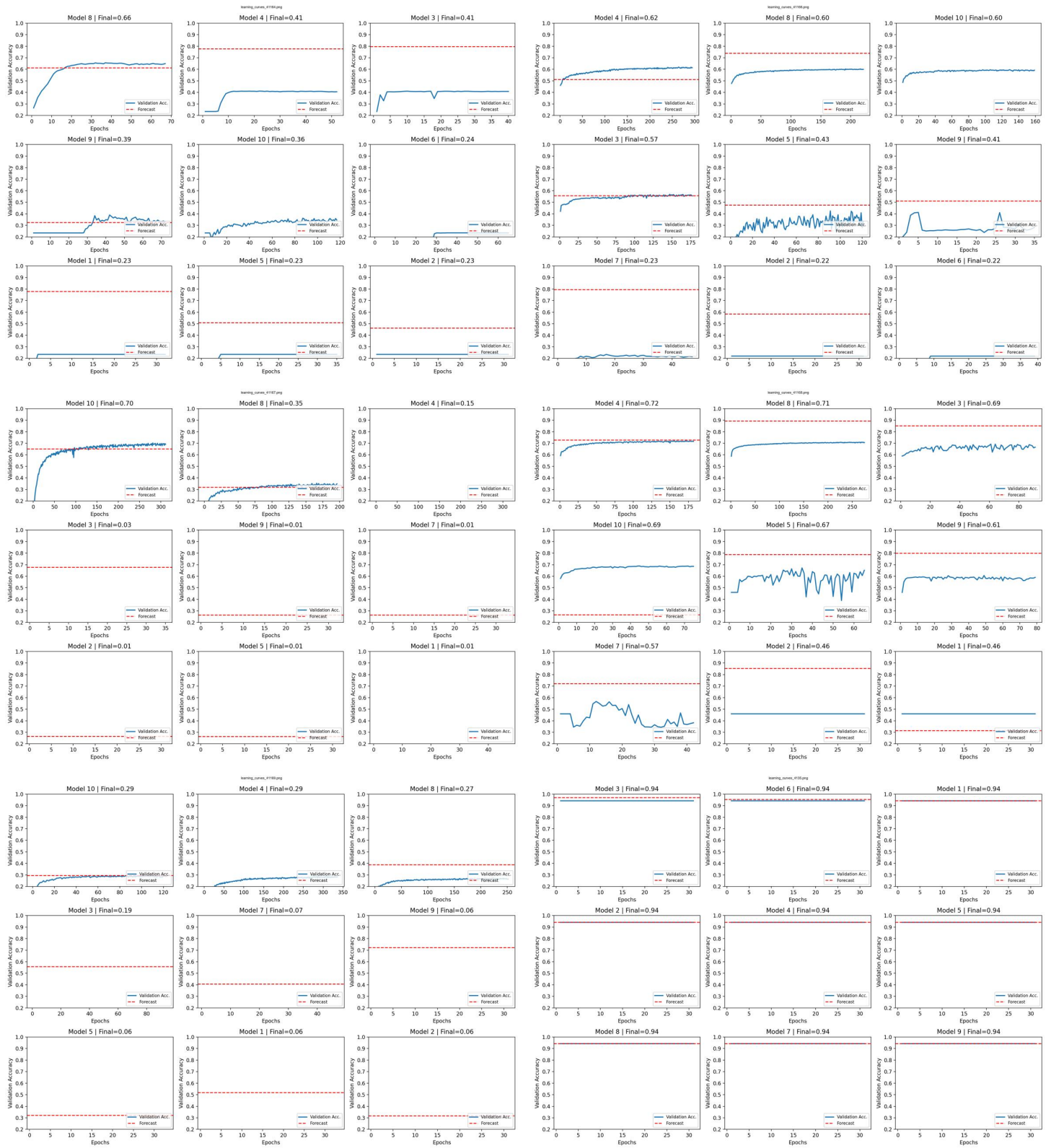


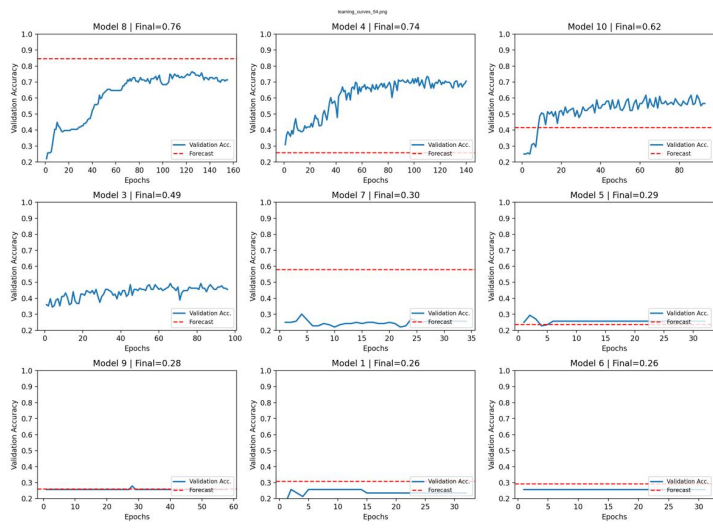




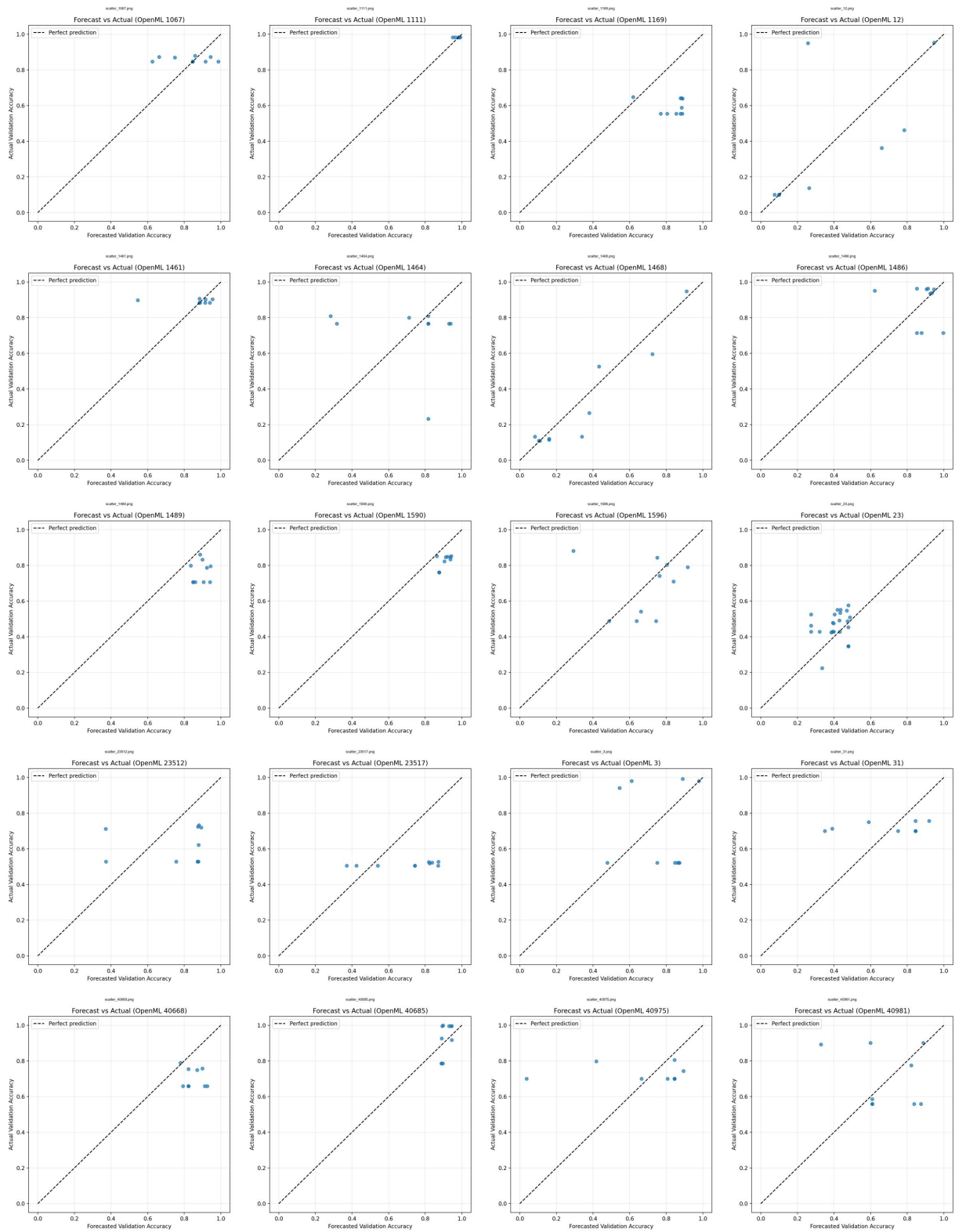


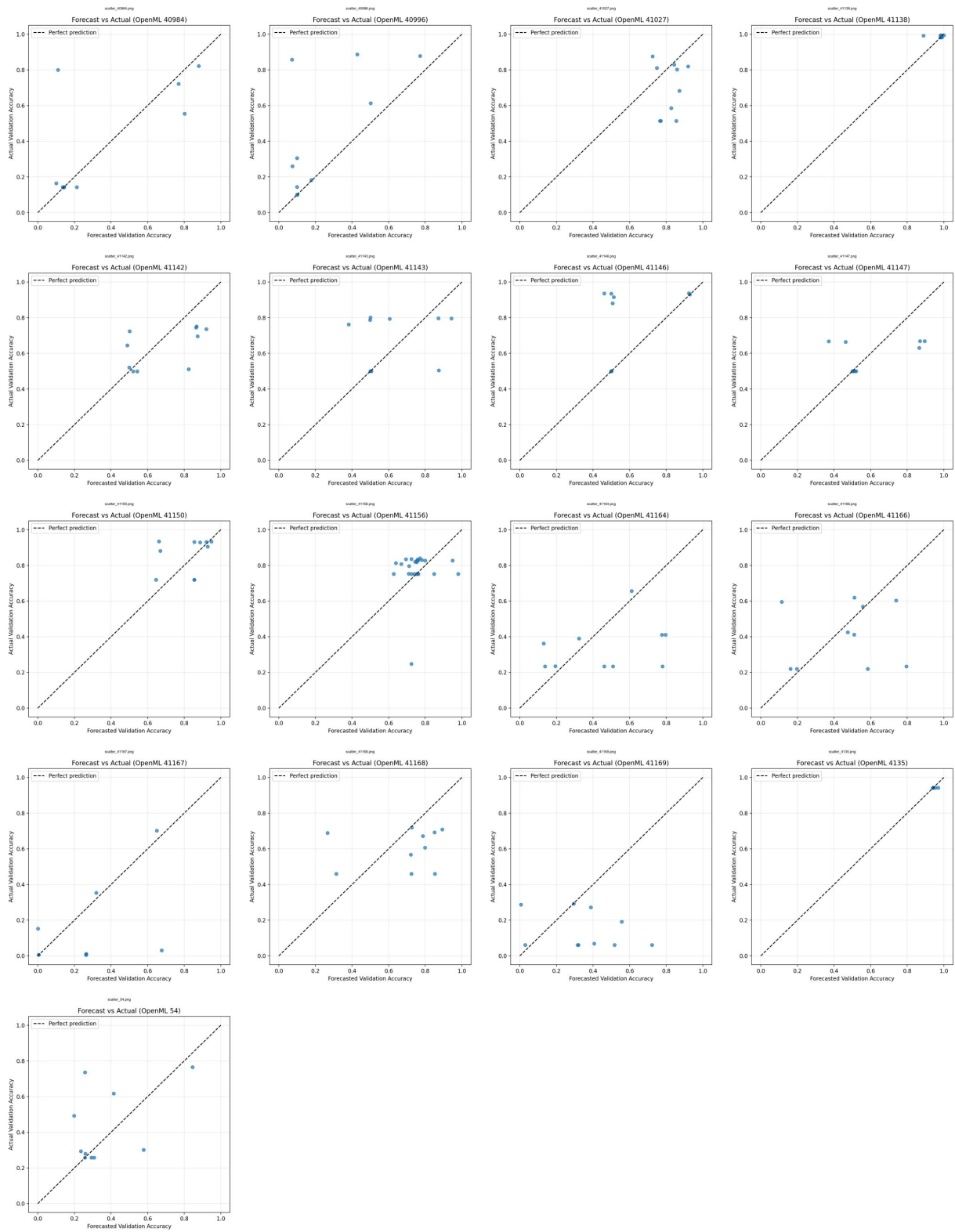




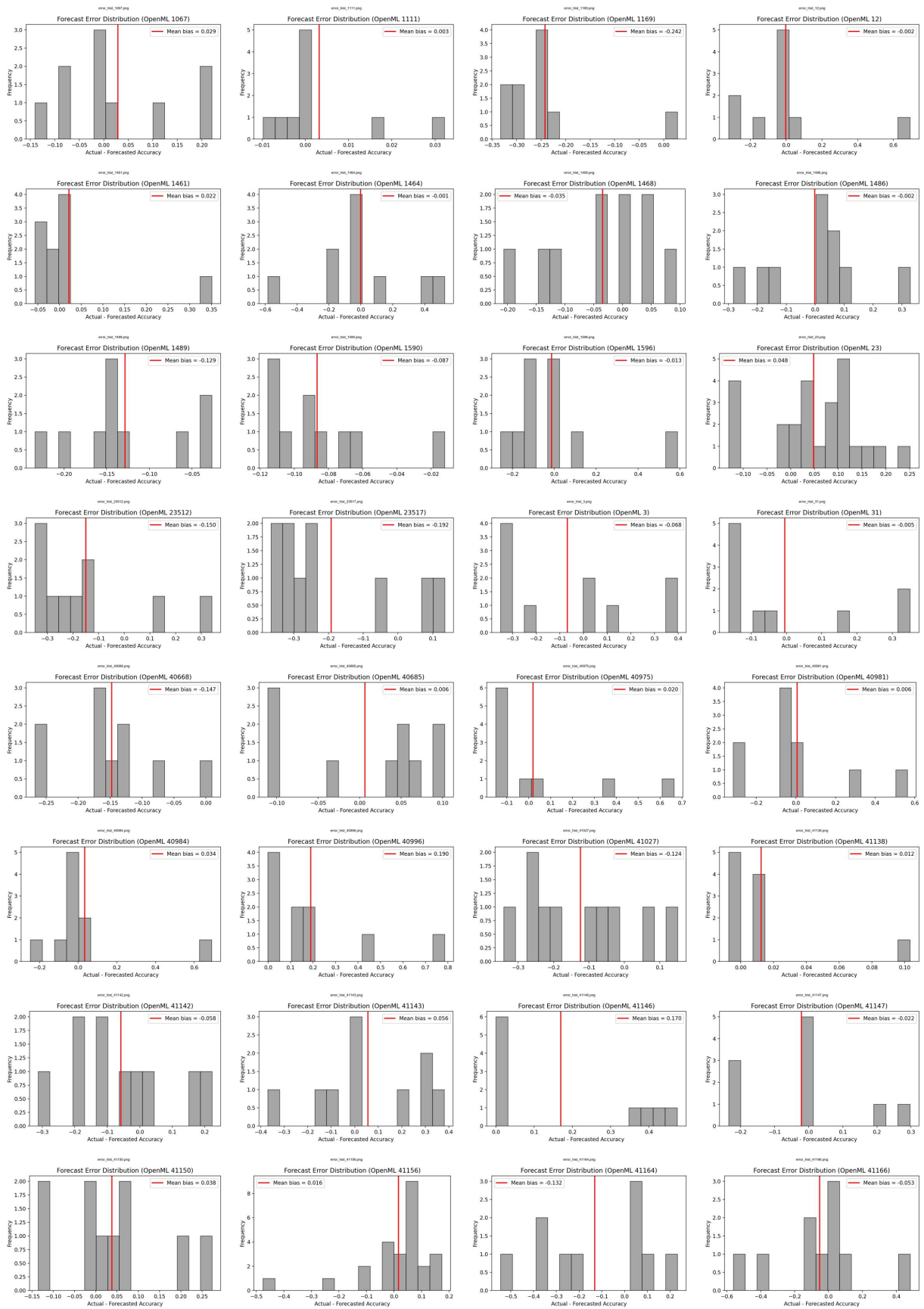


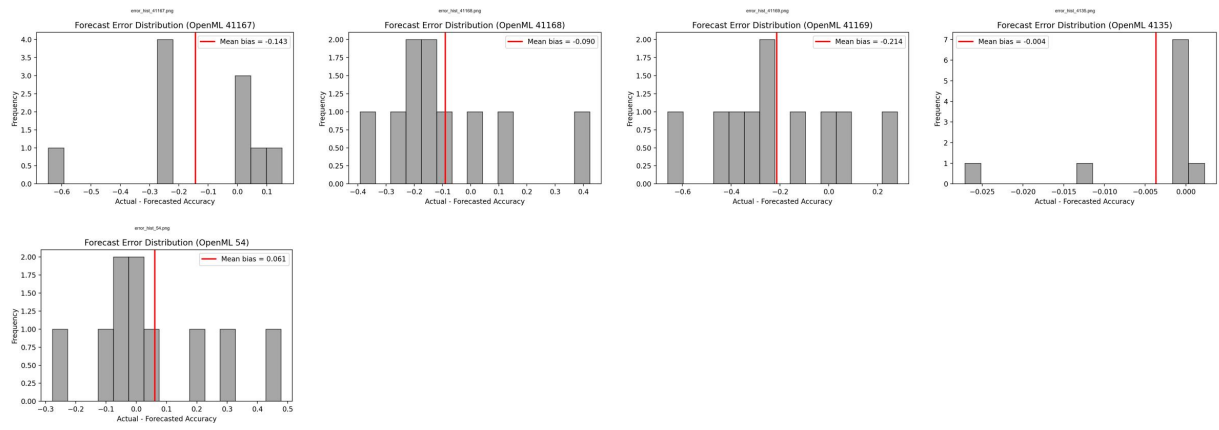
8.4 Appendix D: Scatterplot of the Forecast Performance



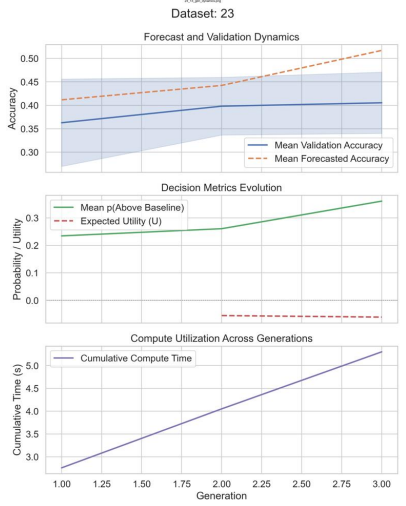
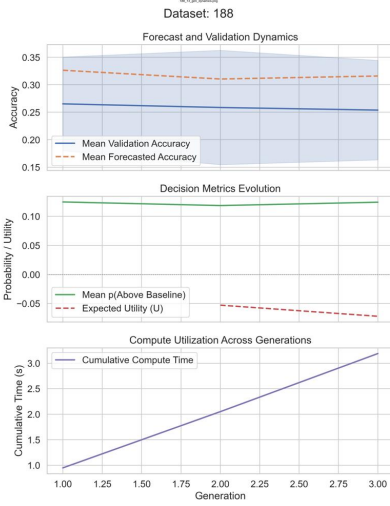
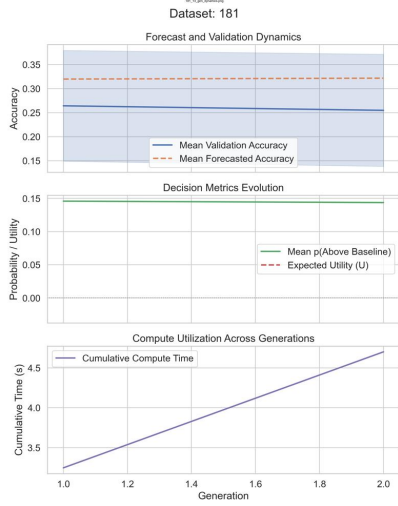
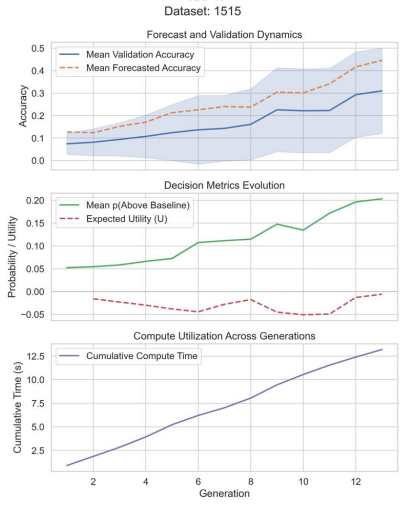
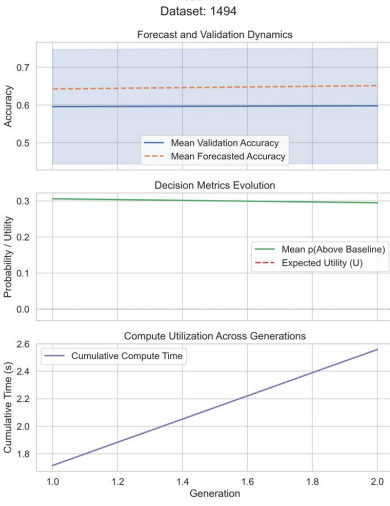
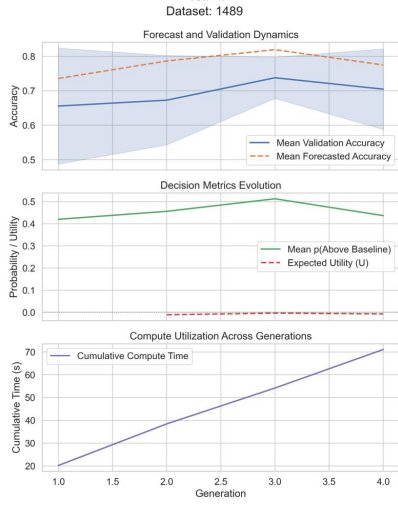
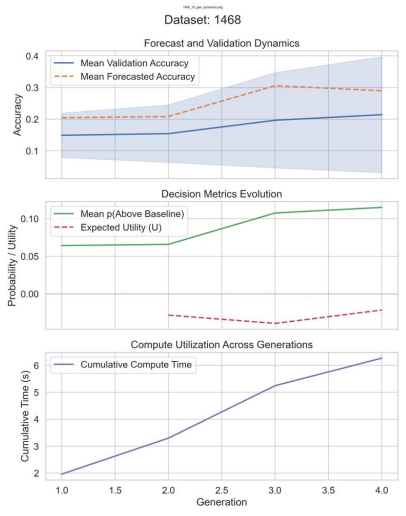
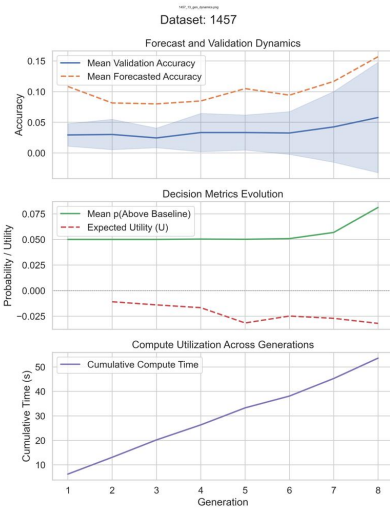
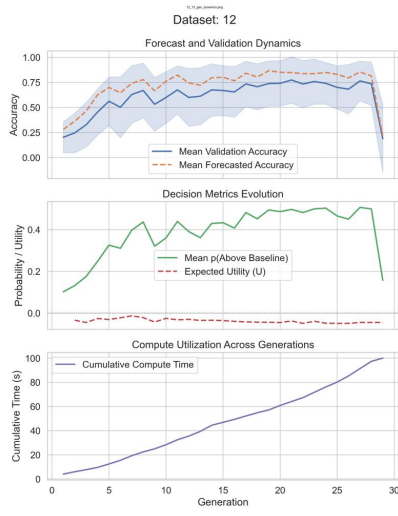


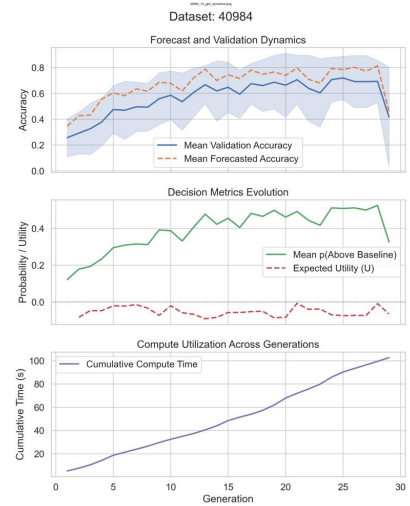
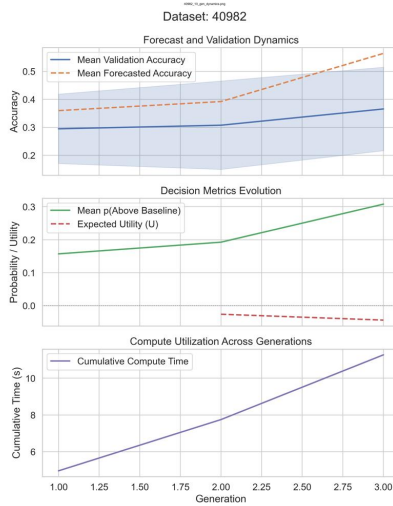
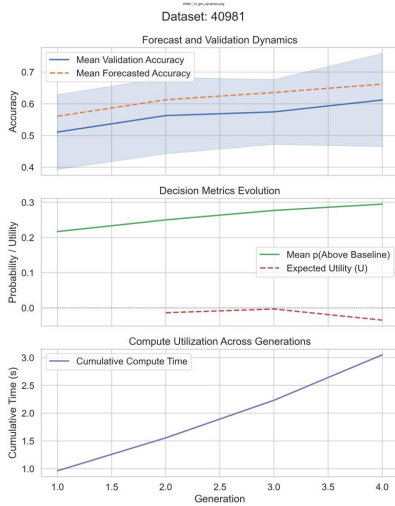
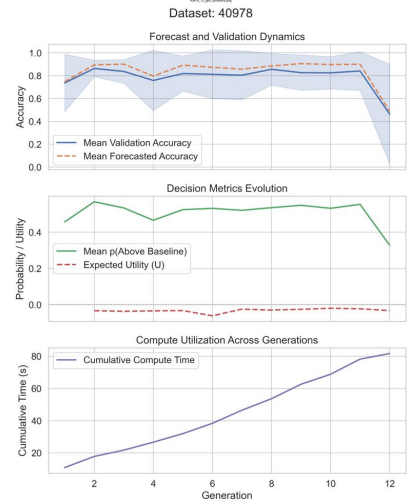
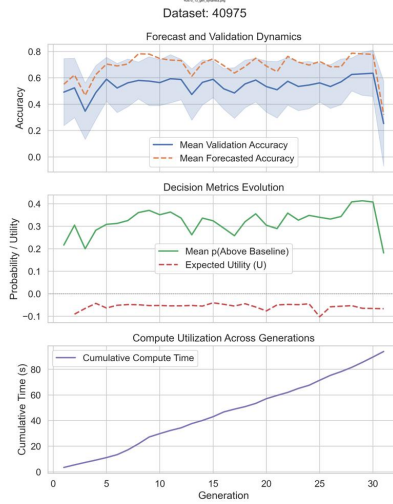
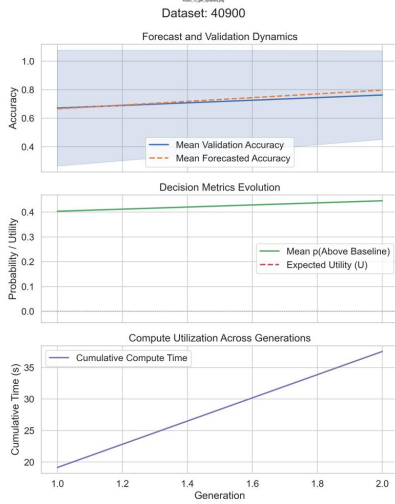
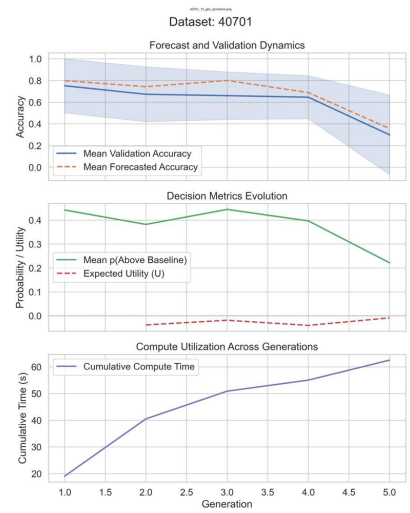
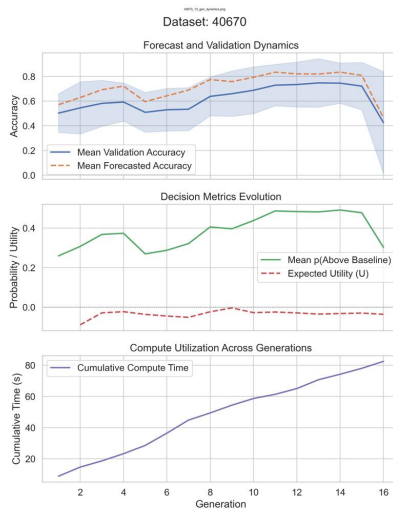
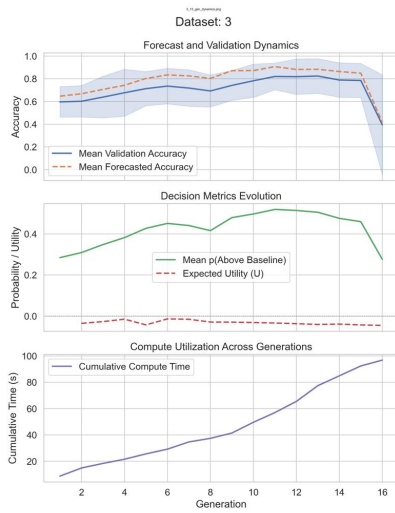
8.5 Appendix E: Error Histogram of the Forecast performance

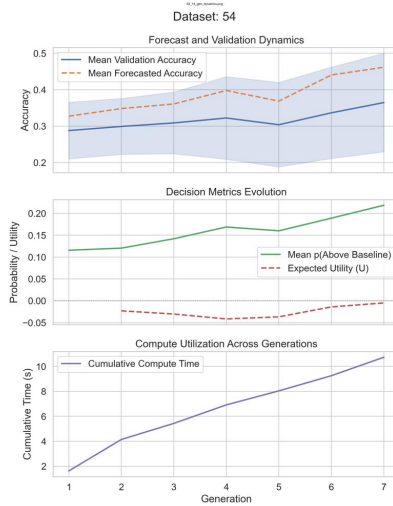
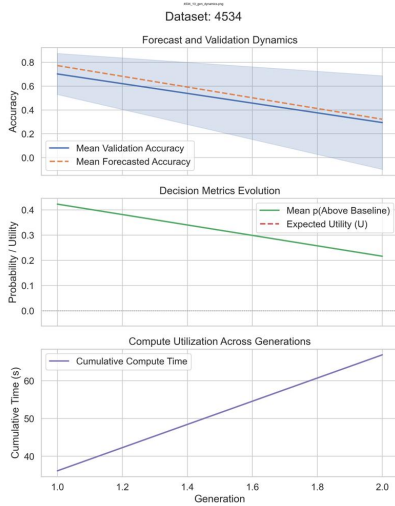
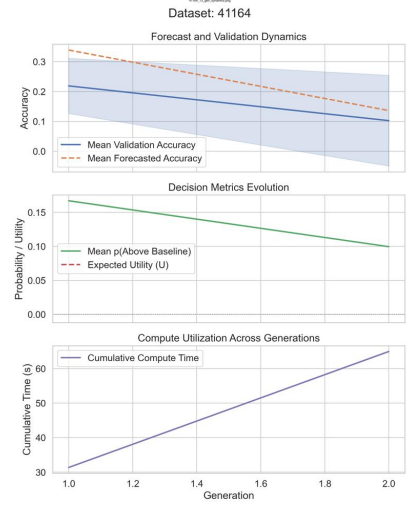
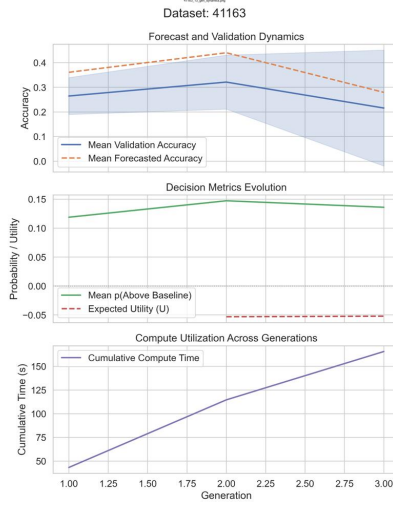
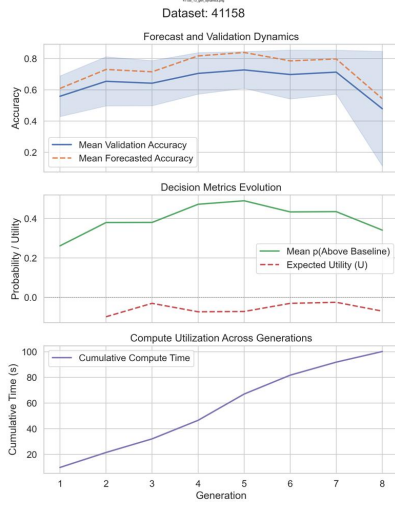
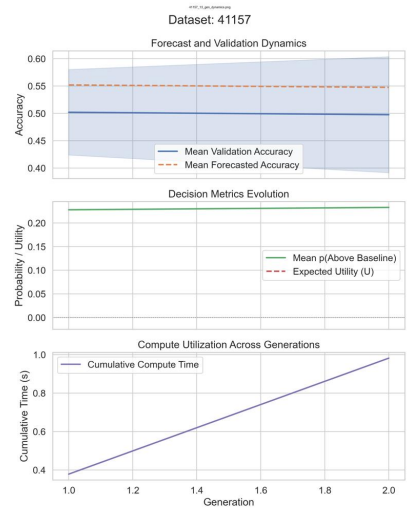
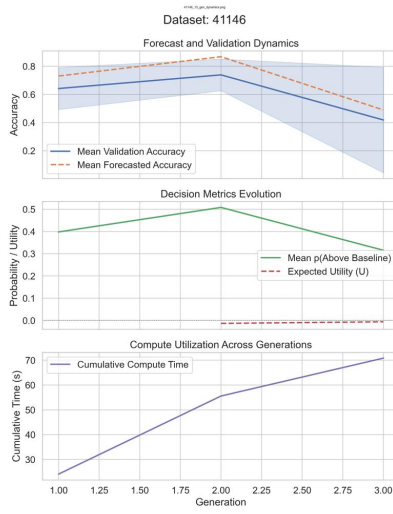
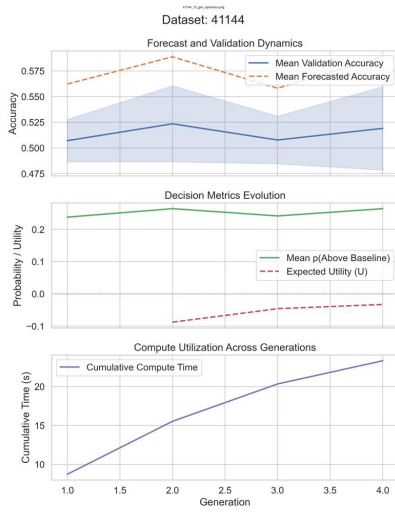




8.6 Appendix F: Should You Even NN Generational Dynamics







8.7 Appendix G: Design Constants and System Parameters

The *ShouldYouEvenNN* framework introduces a set of numerical constants that govern its behaviour. These are not hyperparameters in the sense of dataset-specific tuning; instead, they are fixed **design parameters** chosen once to operationalise the decision process. Their role is to stabilise the training rhythm, regulate the forecasting procedure, and control how the expected-utility function responds to evidence and cost. All values were kept constant across every dataset to preserve transparency and reproducibility.

For clarity, these constants are grouped below according to the component of the framework they influence.

(1) Structural and Training Parameters. These determine how candidate models consume data and progress through the early training phase:

- **Population size (N):** number of architectures evaluated per generation.
- **Starting instances (ρ):** initial fraction of the training data used for the one-epoch check.
- **Growth factor (γ):** multiplicative increase in data usage between generations.

(2) Forecasting Parameters. Regulate the behaviour of the lightweight forecasting mechanism:

- **Forecast horizon multiplier (N_p):** number of projected dataset passes used to estimate the asymptotic accuracy.
- **Forecast smoothing/temperature (T_m):** controls the smoothness of extrapolation when few validation points are available.

(3) Probabilistic Scoring Parameters. Convert the forecast margin into a probability of surpassing the baseline:

- **Sigmoid temperature (t):** controls the steepness of the logistic mapping from forecast margin to confidence.

(4) Decision and Pruning Parameters. Shape the expected-utility calculation and the evolutionary refinement:

- **Cost scale (λ):** weights the contribution of future computational cost.
- **Evidence ramp (τ_e):** controls how quickly evidence gains influence as more batches are observed.
- **Cost compression (τ_c):** smooths large projected training horizons to avoid dominating the utility.
- **Base and maximum drop rates ($\delta_{base}, \delta_{max}$):** govern the overall pruning pressure between generations.
- **Elite fraction (ε):** proportion of the best candidates that are always retained.

Table 8.2: Design constants used in *ShouldYouEvenNN*. All values were fixed once and used uniformly across datasets.

Category	Parameter	Symbol	Default	Purpose
Training	Population size	N	30	Models per generation
Training	Growth factor	γ	1.4	Data expansion per epoch
Forecast	Horizon multiplier	N_p	5	Forecast depth
Forecast	Smoothing temperature	T_m	0.9	Stabilises extrapolation
Scoring	Sigmoid temperature	t	0.05	Smoothness of confidence mapping
Decision	Cost scale	λ	0.05	Weight of future cost
Decision	Evidence ramp	τ_e	8.0	Influence of accumulated evidence
Decision	Cost compression	τ_c	150.0	Dampens large horizons
Evolution	Base drop rate	δ_{base}	0.15	Minimum pruning
Evolution	Maximum drop rate	δ_{max}	0.40	Maximum pruning
Evolution	Elite fraction	ε	0.10	Top models preserved

Remarks. These constants are intended to stabilise behaviour and enforce a consistent decision process rather than to optimise performance on a per-dataset basis. A systematic study of their sensitivity is left for future work.

8.7.1 Reproducibility and Logging

Reproducibility is a central design principle of *ShouldYouEvenNN*. Every experiment, candidate, and decision is recorded deterministically to ensure that identical seeds, configurations, and

datasets produce identical results. This guarantees that any claim of neural competitiveness can be independently verified.

Deterministic Execution. All random operations—from architecture sampling to evolutionary crossover—are governed by synchronized random generators across NumPy, PyTorch, and Python’s internal state. The framework initializes a global seed before each experiment, ensuring that the same sequence of random numbers is used at every stage. Consequently, rerunning the search on the same configuration reproduces the exact same population dynamics, forecasts, and final outcomes.

Structured Logging. Each candidate maintains a comprehensive record of its configuration and performance:

- architectural parameters and initialization seed,
- per-batch training and validation metrics,
- forecasted mean and confidence interval,
- probability of surpassing the baseline,
- expected-utility value and continuation decision,
- refinement statistics and final validation/test accuracy.

These records are serialized into structured `.json` and `.csv` ledgers, enabling direct statistical analysis and visual reconstruction of every run.

Ledger Structure. At the end of each generation, the framework consolidates the state of all candidates into a *ledger*. This ledger captures the full evolution of the population, including forecasts, pruning decisions, and utility scores. Because the ledger is produced deterministically, it functions as both a reproducibility artifact and an audit trail of the decision-making process.

Traceability and Transparency. The combination of deterministic randomness, explicit logging, and structured output ensures that the entire search and decision pipeline can be replayed step by step.